

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CNTT I



BÁO CÁO BÀI TẬP LỚN
BỘ MÔN CÁC HỆ THỐNG PHÂN TÁN

ĐỀ TÀI:

PHÁT TRIỂN HỆ THỐNG SOẠN THẢO VĂN BẢN CỘNG TÁC

Giảng viên: *Kim Ngọc Bách*

Nhóm thực hiện: **03**

Sinh viên thực hiện:

Nguyễn Khắc Gia Hoàng **B22DCCN340**

Đào Danh Đức **B22DCCN220**

Nguyễn Thành Năm **B22DCCN570**

Hà Nội, 2026

PHÂN CÔNG CÔNG VIỆC

STT	Họ và tên	Công việc
1	Nguyễn Khắc Gia Hoàng	- Xây dựng class xử lý vector clock đảm bảo tính nhân quả - Viết hàm biến đổi các thao tác chèn/xóa khi xảy ra tương tranh - Thiết kế schema log lưu trữ và xử lý các biến động để phục vụ việc lưu versioning - Viết chương trình worker consume message từ RabbitMQ để xử lý các thông điệp
2	Nguyễn Thành Năm	- Xây dựng gateway để xử lý các yêu cầu request HTTP, tiếp nhận các kết nối websocket để quản lý phiên, trạng thái của các client. - Cấu hình Exchange/Queue RabbitMQ để luân chuyển thông điệp giữa gateway và worker consumer. - Cài đặt cache lưu trạng thái văn bản hiện tại để đảm bảo đồng bộ văn bản khi có kết nối vào tài liệu của nhiều đang chỉnh sửa.
3	Đào Danh Đức	- Xây dựng giao diện soạn thảo văn bản, hỗ trợ hiển thị đồng bộ chính xác trạng thái văn bản nhiều người dùng cùng lúc. - Cài đặt bộ đếm Vector Clock ở client. Cập nhật bộ đếm Vector Clock mỗi khi có sự kiện gửi/nhận. - Xử lý hiển thị con trỏ thời gian thực và danh sách người dùng online dựa trên luồng sự kiện từ Server - Xây dựng module đăng nhập và phân quyền truy cập (Owner, Editor, Viewer) cho từng tài liệu.

MỤC LỤC

PHÂN CÔNG CÔNG VIỆC	i
MỤC LỤC	ii
THUẬT NGỮ VIẾT TẮT	v
DANH MỤC HÌNH ẢNH.....	vi
DANH MỤC BẢNG BIỂU	vii
LỜI CẢM ƠN.....	viii
CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN PHÁT TRIỂN HỆ THỐNG SOẠN THẢO VĂN BẢN CỘNG TÁC	1
1.1. Mục tiêu nghiên cứu	1
1.2. Yêu cầu hệ thống	2
1.2.1 Yêu cầu chức năng	2
1.2.2 Yêu cầu phi chức năng	3
1.3. Đối tượng và phạm vi nghiên cứu	4
1.3.1. Đối tượng nghiên cứu	4
1.3.2. Phạm vi nghiên cứu	4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT HỆ THỐNG SOẠN THẢO VĂN BẢN CỘNG TÁC	6
2.1. Lý thuyết về tính nhất quán trong hệ thống phân tán.....	6
2.1.1. Khái quát về hệ thống phân tán	6
2.1.2. Định lý CAP và sự đánh đổi kiến trúc.....	7
2.1.3. Tính nhất quán cuối cùng	8

2.2. Các giải pháp đồng bộ và giải quyết xung đột phân tán.....	9
2.2.1. Thuật toán OT.....	10
2.2.2. Thuật toán CRDT	11
2.2.3. So sánh và định hướng lựa chọn giải thuật	13
2.3. Cơ chế kiểm soát quan hệ nhân quả	15
2.3.1. Quan hệ nhân quả Happened-Before.....	15
2.3.2. Thuật toán Vector Clock	16
2.4. Lý thuyết về truyền thông mạng và hạ tầng thời gian thực.....	17
2.4.1. Giao thức truyền thông WebSockets.....	17
2.4.2. Kiến trúc hướng sự kiện và Message Broker	18
2.4.3. Cơ chế bộ nhớ đệm tốc độ cao	18
2.4.4. Hệ quản trị cơ sở dữ liệu phi quan hệ.....	19
CHƯƠNG 3. TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG SOẠN THẢO VĂN BẢN CỘNG TÁC	20
3.1. Kiến trúc hệ thống	20
3.1.1. Các thành phần hạ tầng cốt lõi	20
3.1.2. Luồng hoạt động và xử lý sự kiện	21
3.2. Triển khai backend và thuật toán OT	22
3.2.1. Tầng giao tiếp thời gian thực và cổng điều phối API.....	22
3.2.2. Message Broker Layer (RabbitMQ).....	23
3.2.3. Tầng tiêu thụ và xử lý sự kiện	24
3.2.4. Logic phân giải xung đột và ma trận thuật toán OT.....	25
3.2.5. Tầng bộ nhớ đệm trung tâm	28

3.2.6. Tầng lưu trữ bền vững & đối soát (Persistence Layer với MongoDB).....	31
3.3. Triển khai Frontend	32
3.3.1. Kiến trúc Client và trình soạn thảo phi trạng thái	33
3.3.2. Phát hiện thay đổi cục bộ và thuật toán đồng bộ máy khách	36
3.3.3. Cơ chế hoàn tác cộng tác	40
3.3.4. Hiện thị con trỏ và cơ chế phân quyền	41
3.4. Kết quả triển khai và đánh giá thực nghiệm hệ thống	43
3.4.1. Môi trường và công cụ thực nghiệm	43
3.4.2. Đánh giá tính đúng đắn của thuật toán OT (Correctness Evaluation).....	44
3.4.3. Đánh giá hiệu năng và Khả năng mở rộng (Performance & Scalability Evaluation).....	45
3.4.4. Đánh giá cơ chế xử lý Race Condition khi khôi phục phiên bản	48
3.4.5. Đánh giá trải nghiệm người dùng (UI/UX) và Giao diện tương tác	48
3.5. Đánh giá tổng quan, hạn chế và hướng phát triển	50
3.5.1. Đánh giá tổng quan kết quả đạt được	50
3.5.2. Các hạn chế còn tồn tại.....	51
3.5.3. Hướng phát triển trong tương lai.....	52
KẾT LUẬN	54
TÀI LIỆU THAM KHẢO	56

THUẬT NGỮ VIẾT TẮT

Thuật ngữ viết tắt	Nghĩa tiếng Anh	Nghĩa tiếng Việt
OT	Operational Transformation	Biến đổi thao tác
CAP	Consistency – Availability – Partition Tolerance	Tính nhất quán – Tính sẵn sàng – Khả năng chịu phân hoạch mạng
CRDT	Conflict-free Replicated Data Types.	Kiểu dữ liệu sao chép không xung đột
DOM	Document Object Model	Mô hình Đối tượng Tài liệu
API	Application Programming Interface	Giao diện lập trình ứng dụng
HTTP	Hypertext Transfer Protocol	Giao thức truyền tải siêu văn bản
I/O	Input/Output	Hoạt động đọc/ghi dữ liệu (được nhắc đến ở phần I/O Disk)
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên (nhắc đến rất nhiều ở phần Redis và Client Memory)
UI/UX	User Interface / User Experience	Giao diện người dùng / Trải nghiệm người dùng
WebRTC	Web Real-Time Communication	Giao tiếp thời gian thực trên web (nhắc đến ở phần Hướng phát triển)
LLM	Large Language Model	Mô hình ngôn ngữ lớn (nhắc đến ở phần Hướng phát triển AI)
RBAC	Role-Based Access Control	Kiểm soát truy cập dựa trên vai trò (nếu bạn sử dụng cụm từ này trong phần phân quyền)

DANH MỤC HÌNH ẢNH

Hình 2.1: Mô hình tổng quan của hệ thống phân tán theo kiến trúc Client-Server.....	7
Hình 2.2: Tam giác định lý CAP và mô hình AP của hệ thống.	8
Hình 2.3: Minh họa nguyên lý tính nhất quán cuối cùng trong hệ thống phân tán.	9
Hình 2.4: Minh họa cơ chế biến đổi thao tác trong thuật toán OT.....	11
Hình 2.5: Minh họa cơ chế cấp phát định danh phân số khi chen ký tự mới trong cấu trúc dữ liệu CRDT.	13
Hình 2.6: Minh họa cơ chế Vector Clock trong việc xác định trạng thái nhân quả giữa các sự kiện trong hệ thống phân tán.	17
Hình 3.1: Sơ đồ kiến trúc tổng thể và luồng xử lý sự kiện đồng bộ.....	20
Hình 3.2: Lưu đồ thuật toán mã hóa cấu trúc hiển thị DOM sang chỉ mục logic phẳng. ..	34
Hình 3.3: Lưu đồ thuật toán hai chiều ánh xạ tọa độ giữa DOM và chỉ mục phẳng.....	36
Hình 3.4: Sơ đồ khối thuật toán trích xuất thao tác dựa trên kỹ thuật so sánh chuỗi hai đầu.	38
Hình 3.5: Sơ đồ minh họa cơ chế giải quyết xung đột chen ký tự đồng thời.....	39
Hình 3.6: Mô hình cấu trúc dữ liệu và luồng biến đổi của ngăn xếp hoàn tác cộng tác. ...	41
Hình 3.7: Kết quả chạy kiểm thử tự động (Unit Test) cho các kịch bản phân giải xung đột OT bằng Pytest	45
Hình 3.8: Biểu đồ đối chiếu độ trễ hệ thống theo sự gia tăng của số lượng kết nối tương tranh.	46
Hình 3.9: Giao diện soạn thảo cộng tác với tính năng hiển thị vị trí con trỏ chuột động (Live Cursors)	49
Hình 3.10: Giao diện tính năng xem trước và khôi phục mốc lịch sử phiên bản (Version History).....	50

DANH MỤC BẢNG BIỂU

Bảng 2.1: Bảng so sánh đặc tính kỹ thuật giữa giải thuật OT và CRDT.	14
Bảng 3.1: So sánh độ trễ cuối - cuối (End-to-End Latency) giữa kịch bản xử lý đơn luồng và đa luồng.....	46

LỜI CẢM ƠN

Để hoàn thành toàn bộ kiến trúc hệ thống và bản báo cáo đồ án này, quá trình thực hiện của nhóm đã nhận được sự hỗ trợ vô cùng quý báu từ Thầy. Chúng em xin gửi lời tri ân chân thành và sâu sắc nhất tới Thầy Kim Ngọc Bách, giảng viên trực tiếp phụ trách môn học. Trong suốt thời gian triển khai bài tập lớn, Thầy không chỉ truyền đạt kiến thức nền tảng vững chắc mà còn tận tình giúp chúng em tháo gỡ nhiều nút thắt phức tạp về mặt logic và kỹ thuật.

Những ý kiến đóng góp cùng sự chỉ bảo sát sao của Thầy là kim chỉ nam giúp nội dung báo cáo đi đúng trọng tâm và đảm bảo tính ứng dụng thực tiễn cao. Sự nhiệt tình và tâm huyết của Thầy chính là động lực to lớn để nhóm có thể bám sát tiến độ và hoàn thiện đề tài một cách chín chu nhất. Những bài giảng giá trị trên lớp đã tạo nên nền móng vững chắc để chúng em triển khai thành công các quy trình nghiệp vụ cốt lõi của toàn bộ hệ thống.

Mặc dù đã dành trọn tâm huyết và nỗ lực để xây dựng phần mềm, nhưng do hạn chế về mặt thời gian cũng như kinh nghiệm thực tế, báo cáo chắc chắn vẫn còn tồn đọng những thiếu sót nhất định. Chúng em rất mong sẽ tiếp tục nhận được những ý kiến đánh giá và góp ý chuyên môn từ phía Thầy. Những phản hồi giá trị đó sẽ là hành trang thiết yếu giúp chúng em nâng cao nhận thức và hoàn thiện kỹ năng phát triển phần mềm trong tương lai.

CHƯƠNG 1. GIỚI THIỆU BÀI TOÁN PHÁT TRIỂN HỆ THỐNG SOẠN THẢO VĂN BẢN CỘNG TÁC

1.1. Mục tiêu nghiên cứu

Mục tiêu xuyên suốt của nghiên cứu này là thiết kế và xây dựng thành công một hệ thống soạn thảo văn bản cộng tác theo thời gian thực. Hệ thống hướng tới việc giải quyết bài toán đồng bộ dữ liệu khi có nhiều người dùng ở các vị trí khác nhau cùng truy cập và chỉnh sửa trên một tài liệu. Yêu cầu đặt ra là phải đảm bảo không xảy ra hiện tượng xung đột hay mất mát thông tin, duy trì độ trễ thấp và mang lại trải nghiệm tương tác mượt mà tương tự như các nền tảng thương mại hiện nay.

Về mặt lý thuyết và giải thuật, trọng tâm được đặt vào việc đi sâu tìm hiểu cách xử lý xung đột dữ liệu thông qua giải thuật OT. Thay vì khóa tài liệu khi một người đang gõ, hệ thống sẽ cho phép tất cả cộng tác viên chỉnh sửa tự do cùng lúc. Mục tiêu là áp dụng thuật toán này để tự động tính toán, biến đổi các thao tác gõ phím hoặc xóa chữ từ nhiều nguồn gửi đến, sao cho kết quả hiển thị trên màn hình của mọi người dùng cuối cùng đều hoàn toàn đồng nhất.

Để hiện thực hóa thuật toán trên, việc xây dựng một kiến trúc phần mềm hướng sự kiện vững chắc và có khả năng mở rộng là mục tiêu then chốt tiếp theo. Hệ thống sẽ kết hợp linh hoạt các công nghệ hiện đại như WebSockets để truyền tải dữ liệu tức thì, cùng với hàng đợi thông điệp RabbitMQ, bộ nhớ đệm Redis và cơ sở dữ liệu MongoDB. Cụm công nghệ này đóng vai trò phân luồng dữ liệu hợp lý, giúp máy chủ có thể xử lý mượt mà hàng loạt thao tác gõ phím liên tục mà không bị quá tải hay sai lệch thứ tự.

Cuối cùng, về mặt ứng dụng thực tiễn, quá trình phát triển sẽ hoàn thiện một ứng dụng web với giao diện người dùng trực quan bằng Next.js. Ứng dụng cung cấp đầy đủ các tính năng như quản lý danh sách tài liệu, phân quyền và đặc biệt là khả năng theo dõi vị trí con trỏ chuột của người khác theo thời gian thực. Đồng thời, việc kiểm thử tự động thuật toán và đánh giá hiệu năng cũng được thực hiện nhằm chứng minh tính khả thi và độ ổn định của toàn bộ kiến trúc đã đề ra.

1.2. Yêu cầu hệ thống

Để hiện thực hóa được những mục tiêu nghiên cứu đã đề ra và đảm bảo ứng dụng vận hành ổn định trong môi trường thực tế, nghiên cứu này xác định rõ các tiêu chuẩn kỹ thuật và tính năng cốt lõi cần phải đáp ứng. Các yêu cầu này được chia thành hai nhóm chính: yêu cầu chức năng quy định những tác vụ người dùng có thể thực hiện, và yêu cầu phi chức năng quy định các giới hạn kỹ thuật ẩn bên dưới hệ thống.

1.2.1 Yêu cầu chức năng

a. Quản lý tài khoản và lưu trữ tài liệu

Đầu tiên, hệ thống phải cung cấp cơ chế xác thực người dùng và bảo mật. Ứng dụng cho phép người dùng đăng ký, đăng nhập để truy cập vào kho dữ liệu riêng của từng tài khoản. Tại giao diện quản lý trung tâm, người dùng có thể thực hiện các thao tác cơ bản với cơ sở dữ liệu như thêm mới, đổi tên, xóa. Bên cạnh đó, hệ thống cần truy xuất và hiển thị các thông tin đi kèm của tài liệu như thời gian tạo, lần cập nhật cuối cùng và danh sách các tài liệu được người khác chia sẻ, giúp người dùng phân loại và quản lý dữ liệu hiệu quả.

b. Soạn thảo cộng tác không khóa dòng và xử lý xung đột

Đây là chức năng cốt lõi của hệ thống. Khác với các hệ thống quản lý tệp tin truyền thống thường phải khóa tệp khi có người đang chỉnh sửa, hệ thống này yêu cầu một cơ chế chỉnh sửa tự do. Nó phải cho phép nhiều trình duyệt cùng kết nối vào một tài liệu và tiến hành gõ phím, chèn hoặc xóa chữ đồng thời ở bất kỳ vị trí nào. Để làm được điều này, trình soạn thảo tại phía người dùng phải nhận diện sự kiện bàn phím, gửi luồng dữ liệu về máy chủ và phân giải xung đột theo thời gian thực. Hệ thống phải đảm bảo mọi thao tác gõ phím đều được giữ nguyên vẹn, không xảy ra hiện tượng ghi đè dữ liệu hay xô lệch cấu trúc văn bản ngay cả khi nhiều thao tác từ các người dùng khác nhau gửi lên Server trong cùng một phần nghìn giây.

c. Hiển thị trạng thái hoạt động và phân quyền bảo mật

Trong môi trường làm việc nhóm, việc theo dõi các hoạt động của người khác là yếu tố quan trọng để tránh chỉnh sửa trùng lặp. Do đó, hệ thống yêu cầu chức năng hiển thị

trạng thái kết nối trực tuyến. Người dùng sẽ nhìn thấy danh sách các tài khoản đang mở tài liệu, đồng thời theo dõi được vị trí con trỏ chuột cập nhật liên tục trên màn hình. Đi kèm với tính năng này là cơ chế kiểm soát truy cập chặt chẽ. Người tạo tài liệu có quyền thiết lập vai trò cho từng tài khoản được mời, bao gồm quyền chỉnh sửa hoặc quyền chỉ xem. Việc kiểm tra quyền này phải được xác thực trực tiếp tại tầng máy chủ nhằm chặn đứng và từ chối mọi yêu cầu sửa đổi trái phép từ các tài khoản không có thẩm quyền.

1.2.2 Yêu cầu phi chức năng

a. Tính nhất quán cuối cùng

Trong môi trường mạng phân tán, các gói tin truyền đi thường xuyên gặp phải tình trạng trễ mạng hoặc đến Server sai thứ tự so với thời điểm gõ phím thực tế. Vì vậy, tính nhất quán cuối cùng là tiêu chuẩn kỹ thuật bắt buộc. Yêu cầu đặt ra là bất kể sự chênh lệch về tốc độ đường truyền của từng người dùng, hay việc các sự kiện gõ phím bị xáo trộn, thuật toán tính toán tại máy chủ vẫn phải biến đổi chính xác các vị trí index. Hệ thống phải đảm bảo rằng ngay sau khi tất cả người dùng ngừng thao tác, văn bản hiển thị trên màn hình của tất cả các Client và bản ghi lưu trong cơ sở dữ liệu phải hội tụ về một trạng thái duy nhất và hoàn toàn khớp nhau.

b. Độ trễ thấp và khả năng mở rộng

Việc sử dụng các giao thức gọi API truyền thống như HTTP REST là không đủ để đáp ứng tính năng thời gian thực. Hệ thống bắt buộc phải tích hợp giao thức kết nối hai chiều liên tục WebSockets để tối ưu hóa thời gian phản hồi. Thời gian từ lúc một thao tác gõ được tạo ra tại một máy, truyền qua mạng, xử lý tại máy chủ và phát đi lên màn hình của các Client khác phải được giữ ở mức dưới thấp nhất. Cuối cùng, hệ thống cần được thiết kế theo kiến trúc hướng sự kiện, sử dụng hàng đợi thông điệp để điều phối luồng dữ liệu. Khi số lượng tài liệu đang hoạt động và lượng kết nối WebSocket tăng cao, hệ thống phải có khả năng phân bổ tải tính toán ra nhiều tiến trình chạy song song mà không gây quá tải tài nguyên phần cứng.

1.3. Đối tượng và phạm vi nghiên cứu

Để đảm bảo tính tập trung và tính khả thi trong quá trình triển khai, nghiên cứu này xác định rõ các đối tượng cần tìm hiểu sâu về mặt lý thuyết, đồng thời vạch ra những giới hạn cụ thể về mặt chức năng và công nghệ của hệ thống phần mềm sẽ được xây dựng.

1.3.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu trọng tâm của đề tài là nền tảng lý thuyết về các giải thuật đồng bộ dữ liệu trong hệ thống phân tán, mà cốt lõi là thuật toán OT. Trọng điểm nằm ở việc phân tích toán học và xây dựng ma trận biến đổi cho các thao tác tác động lên chuỗi ký tự nhằm giải quyết xung đột khi có nhiều thao tác xảy ra cùng lúc.

Bên cạnh giải thuật cốt lõi, nghiên cứu cũng đi sâu vào việc phân tích cơ chế truyền tải gói tin mạng theo thời gian thực thông qua giao thức WebSockets. Đồng thời, mô hình kiến trúc phần mềm hướng sự kiện cũng là một đối tượng nghiên cứu quan trọng. Quá trình này bao gồm việc khảo sát cách thức kết hợp các thành phần trung gian như hàng đợi thông điệp định tuyến tuần tự RabbitMQ, bộ nhớ đệm truy xuất tốc độ cao Redis và hệ quản trị cơ sở dữ liệu phi quan hệ MongoDB để tạo thành một hệ thống xử lý Backend có khả năng chịu lỗi và phân bổ tải tính toán.

1.3.2. Phạm vi nghiên cứu

Phạm vi về chức năng ứng dụng về mặt tính năng, hệ thống được giới hạn trong việc giải quyết triệt để bài toán đồng bộ và xử lý xung đột cho định dạng văn bản cơ bản, kết hợp với các chức năng kiểm soát phiên làm việc như hiển thị vị trí con trỏ chuột thời gian thực và quản lý quyền truy cập tài liệu. Ở giai đoạn này, hệ thống chưa mở rộng phạm vi sang việc xử lý các định dạng văn bản nâng cao như chèn hình ảnh đa phương tiện, nhúng video, hoặc vẽ bảng biểu phức tạp. Nghiên cứu cũng chưa bao gồm các tính năng hỗ trợ giao tiếp phụ trợ như gọi thoại hay nhắn tin trực tiếp bên trong phòng soạn thảo.

Phạm vi về môi trường và công nghệ triển khai về giới hạn nền tảng, hệ thống được xây dựng dưới dạng ứng dụng nền web tuân theo mô hình Client-Server. Phía máy khách được giới hạn phát triển bằng JavaScript/TypeScript với framework Next.js, hoạt động trên

các trình duyệt web phổ biến. Phía máy chủ được phát triển bằng ngôn ngữ Python sử dụng framework FastAPI để xử lý các luồng bất đồng bộ. Việc kiểm chứng tính đúng đắn của giải thuật OT được khoanh vùng kiểm thử tự động tại tầng Backend thông qua thư viện Pytest, tập trung vào việc mô phỏng các kịch bản xung đột dữ liệu từ nhiều Client trước khi tích hợp lên giao diện người dùng.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT HỆ THỐNG SOẠN THẢO VĂN BẢN CỘNG TÁC

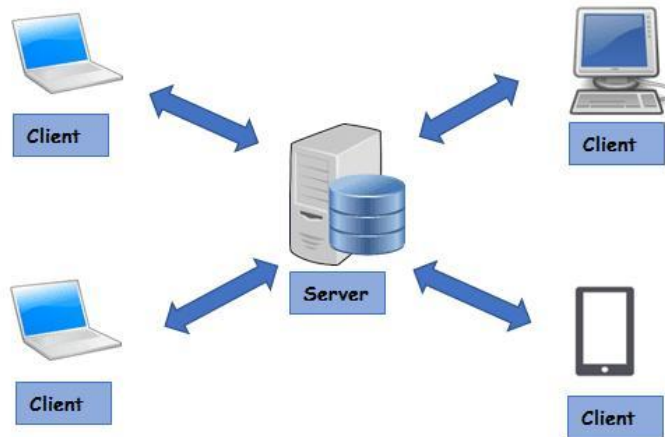
2.1. Lý thuyết về tính nhất quán trong hệ thống phân tán

2.1.1. *Khái quát về hệ thống phân tán*

Hệ thống phân tán là một tập hợp các thiết bị tính toán độc lập, được kết nối với nhau thông qua mạng truyền thông. Các thiết bị này phối hợp hoạt động bằng cách truyền tải thông điệp để hoàn thành một mục tiêu chung, khiến người dùng cảm nhận như đang tương tác với một khối thống nhất. Trong bối cảnh của ứng dụng soạn thảo cộng tác, trình duyệt của mỗi người dùng đóng vai trò Client và các máy chủ xử lý đóng vai trò Server chính là các nút mạng trong hệ thống phân tán này.

Đặc trưng cơ bản của hệ thống phân tán mang lại nhiều thách thức về mặt kỹ thuật. Thứ nhất là tính đồng thời, trong đó nhiều nút mạng có thể thực thi các tác vụ độc lập cùng một lúc, dẫn đến hiện tượng tranh chấp tài nguyên hoặc xung đột trạng thái. Thứ hai là việc không có đồng hồ chung, khiến hệ thống không thể dựa vào thời gian thực tế để sắp xếp chính xác thứ tự trước sau của các sự kiện truyền qua mạng. Cuối cùng là tính lỗi độc lập, nghĩa là một thành phần trong hệ thống có thể bị mất kết nối hoặc ngừng hoạt động đột ngột mà không làm sập toàn bộ mạng lưới.

Về mặt thiết kế tổng thể, kiến trúc hệ thống phân tán có thể được triển khai dưới nhiều mô hình đa dạng như Client-Server, Peer-to-Peer hoặc các hệ thống hiện đại như Microservices. Mỗi mô hình đều có những điểm mạnh và giới hạn riêng trong việc xử lý luồng dữ liệu mạng. Trong phạm vi nghiên cứu này, hệ thống được xây dựng dựa trên nền tảng kiến trúc Client-Server. Ở đó, các Client sẽ thiết lập và duy trì kết nối tới một cụm Server trung tâm. Máy chủ lúc này đóng vai trò trung tâm điều phối, chịu trách nhiệm tiếp nhận các thao tác chỉnh sửa đồng thời, kiểm soát tính hợp lệ, xử lý giải thuật đồng bộ và cuối cùng là phân phối lại luồng dữ liệu trạng thái mới nhất cho tất cả các Client để duy trì một tài liệu dùng chung duy nhất.



Hình 2.1: Mô hình tổng quan của hệ thống phân tán theo kiến trúc Client-Server.

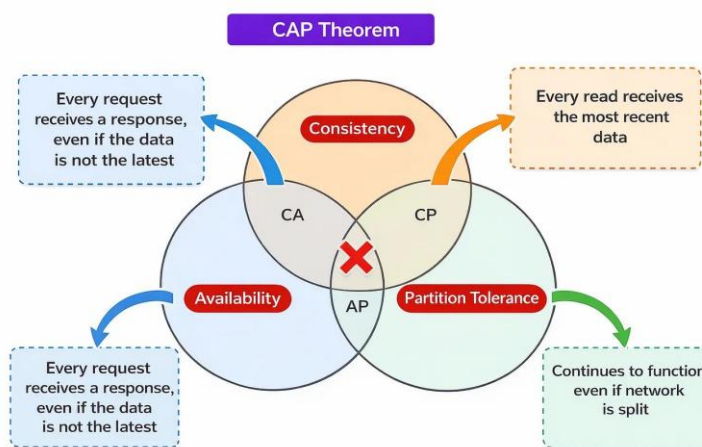
2.1.2. Định lý CAP và sự đánh đổi kiến trúc

Định lý CAP, được đề xuất bởi nhà khoa học máy tính Eric Brewer, là một nguyên lý nền tảng đóng vai trò kim chỉ nam trong việc thiết kế các hệ thống cơ sở dữ liệu và kiến trúc phân tán. Định lý này khẳng định một cách toán học rằng một hệ thống lưu trữ dữ liệu phân tán không thể đồng thời đảm bảo trọn vẹn cả ba thuộc tính cốt lõi Consistency (C), Availability (A) và Partition Tolerance (P). Cụ thể, Consistency yêu cầu mọi nút mạng phải trả về cùng một dữ liệu mới nhất tại bất kỳ thời điểm truy xuất nào, tính sẵn sàng đảm bảo mọi yêu cầu gửi đến hệ thống đều nhận được phản hồi thành công hoặc thất bại mà không bị treo quá thời gian quy định timeout, và Partition Tolerance đảm bảo hệ thống vẫn tiếp tục vận hành ngay cả khi có sự cố mất kết nối hoặc rớt gói tin giữa các cụm máy chủ.

Khi triển khai các hệ thống web trên môi trường Internet thực tế, việc xảy ra độ trễ mạng hoặc gián đoạn kết nối là điều tất yếu không thể tránh khỏi. Do đó, việc đảm bảo thuộc tính P luôn là điều kiện bắt buộc đối với mọi hệ thống phân tán. Từ ràng buộc này, quá trình thiết kế hệ thống phải đối mặt với sự đánh đổi kiến trúc giữa hai mô hình còn lại là CP và AP. Việc lựa chọn mô hình nào phụ thuộc hoàn toàn vào bài toán nghiệp vụ đặc thù và yêu cầu về hiệu năng của từng ứng dụng.

Khi áp dụng định lý CAP vào bài toán xây dựng hệ thống soạn thảo văn bản cộng tác theo thời gian thực, kiến trúc hệ thống bắt buộc phải đưa ra sự đánh đổi rõ ràng. Nếu

lựa chọn mô hình CP, hệ thống sẽ ưu tiên dữ liệu nhất quán tuyệt đối giữa các nút trước khi phản hồi thao tác của người dùng. Tuy nhiên, trong ứng dụng soạn thảo thời gian thực, cách tiếp cận này đòi hỏi việc triển khai các cơ chế khóa đồng bộ đối với văn bản, tạo ra độ trễ rất cao khi nhập liệu và làm giảm nghiêm trọng trải nghiệm thao tác liên tục. Vì vậy, hệ thống bắt buộc phải ưu tiên Availability hơn Consistency, định hướng thiết kế theo mô hình AP. Việc áp dụng mô hình AP cho phép các trình duyệt máy khách ghi nhận ngay lập tức thao tác gõ phím vào bộ đệm cục bộ, duy trì luồng nhập liệu mượt mà mà không cần chờ xác thực vòng lặp từ máy chủ trung tâm.



Hình 2.2: Tam giác định lý CAP và mô hình AP của hệ thống.

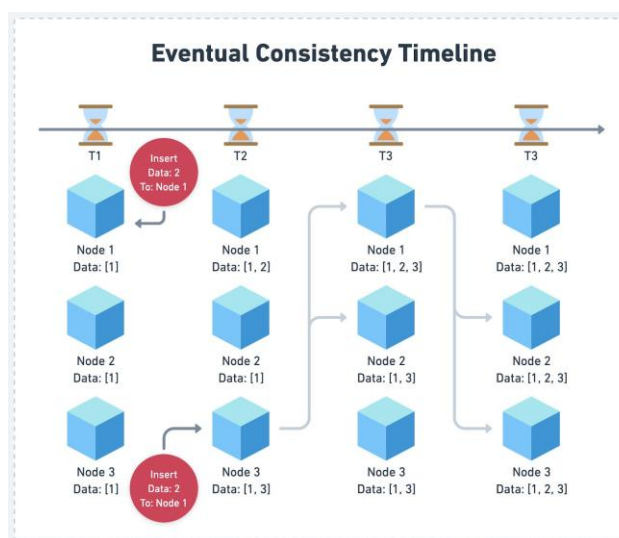
2.1.3. Tính nhất quán cuối cùng

Lựa chọn kiến trúc AP buộc hệ thống phải từ bỏ tính nhất quán tức thời và chuyển sang áp dụng mô hình Tính nhất quán cuối cùng. Đây là một cơ chế đồng bộ nói lỏng, không yêu cầu dữ liệu trên toàn bộ mạng lưới phải trùng khớp tại cùng một phần nghìn giây, mà chấp nhận một khoảng thời gian trễ nhất định để truyền tải các gói tin cập nhật. Nguyên lý cốt lõi của mô hình này đảm bảo rằng, nếu không có thêm bất kỳ sự kiện ghi nào phát sinh, toàn bộ các bản sao dữ liệu trên các máy khách và máy chủ trung tâm cuối cùng sẽ tự động hội tụ về một trạng thái duy nhất và chính xác.

Trong các hệ thống phân tán truyền thống, quá trình đồng bộ thường được giải quyết bằng cơ chế thao tác ghi cuối được ưu tiên dựa trên nhãn thời gian. Tuy nhiên, đối với ứng

dụng soạn thảo cộng tác theo thời gian thực, phương pháp này hoàn toàn không khả thi vì sẽ trực tiếp gây ra hiện tượng ghi đè dữ liệu. Việc ghi đè không chỉ làm mất mát các thao tác hợp lệ của những người dùng có độ trễ mạng cao, mà còn khiến trạng thái văn bản trên màn hình của các trình duyệt liên tục phân kỳ khi các sự kiện chèn hoặc xóa ký tự diễn ra đan xen tại cùng một chỉ số.

Để hiện thực hóa tiêu chuẩn Tính nhất quán cuối cùng trong môi trường chỉnh sửa cộng tác nhiều người dùng, hệ thống tích hợp một cơ chế xử lý xung đột chuyên biệt mang tên giải thuật OT. Khác với các phương pháp xử lý thụ động, giải thuật này chủ động tính toán và biến đổi các chỉ số vị trí của những thao tác gửi đến trễ trước khi áp dụng chúng lên tài liệu gốc. Quá trình biến đổi toán học này đảm bảo rằng dù thứ tự nhận gói tin của các Client bị xáo trộn do độ trễ mạng, mọi nút mạng cuối cùng vẫn kết xuất ra một chuỗi văn bản đồng nhất, bảo toàn trọn vẹn ý định sửa đổi của tất cả cộng tác viên tham gia.



Hình 2.3: Minh họa nguyên lý tính nhất quán cuối cùng trong hệ thống phân tán.

2.2. Các giải pháp đồng bộ và giải quyết xung đột phân tán

Giải quyết xung đột dữ liệu là bài toán then chốt trong hệ thống soạn thảo cộng tác. Khi nhiều thao tác chỉnh sửa diễn ra đồng thời tại cùng một vị trí trên văn bản, việc áp dụng cơ chế ghi đè đơn thuần sẽ làm xô lệch toàn bộ chỉ số và phá vỡ cấu trúc câu chữ. Hiện nay, giới công nghệ chủ yếu tiếp cận và giải quyết bài toán này thông qua hai nhóm giải thuật

cốt lõi là OT và CRDT

2.2.1. Thuật toán OT

Operational Transformation là một họ giải thuật toán học được thiết kế chuyên biệt để hỗ trợ đồng bộ dữ liệu, cho phép nhiều người dùng cùng chỉnh sửa văn bản mà không cần áp dụng cơ chế khóa dòng. Thay vì thay đổi cấu trúc lưu trữ tĩnh của tài liệu, giải thuật này tập trung vào việc biến đổi chính các thao tác tác động lên tài liệu đó. Để hiểu rõ cơ chế vận hành, giải thuật này được phân tích qua ba khía cạnh cốt lõi sau

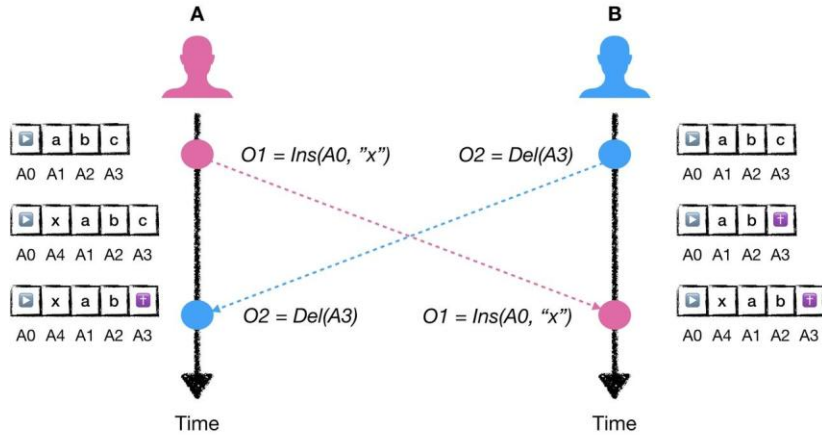
a. Khái niệm và các thành phần thao tác cơ bản

Trong mô hình OT, mọi thay đổi đối với văn bản không được gửi đi dưới dạng một chuỗi ký tự tĩnh, mà được đóng gói thành các đối tượng thao tác. Một thao tác hoàn chỉnh thường được cấu thành từ ba hành động cơ bản: chèn ký tự, xóa ký tự và dịch chuyển con trỏ. Nguyên lý trung tâm của OT là khi một thao tác gõ phím được truyền đi trễ do sự cố mạng, tài liệu đích trên máy chủ có thể đã bị thay đổi bởi thao tác của một máy khách khác. Thay vì từ chối thao tác đến trễ hoặc ghi đè dữ liệu một cách máy móc, hệ thống sẽ tiếp nhận thao tác này và đưa vào luồng xử lý biến đổi chỉ số.

b. Cơ chế hàm biến đổi thao tác

Hàm biến đổi là hạt nhân logic của toàn bộ giải thuật OT. Giả sử hai trình duyệt Client A và Client B cùng tạo ra thao tác O1 và O2 tại cùng một trạng thái gốc của văn bản. Do chênh lệch tốc độ đường truyền, máy chủ nhận được và thực thi O1 trước. Khi luồng dữ liệu O2 từ Client B đến sau, hệ thống không thể áp dụng trực tiếp O2 lên văn bản vì chỉ số vị trí các ký tự đã bị thay đổi hoàn toàn bởi tác động của O1.

Lúc này, máy chủ sẽ kích hoạt hàm biến đổi toán học $T(O2, O1)$ để tính toán ra một thao tác mới là O2 phẩy. Quá trình dịch chuyển chỉ số này tính toán độ lệch vị trí do O1 gây ra, từ đó cập nhật lại tọa độ đích cho O2. Kết quả là khi O2 phẩy được áp dụng lên văn bản, ký tự của Client B vẫn được chèn hoặc xóa ở đúng ngữ cảnh vị trí mà người dùng mong muốn ban đầu, bảo toàn tính toàn vẹn của dữ liệu đồng thời.



Hình 2.4: Minh họa cơ chế biến đổi thao tác trong thuật toán OT.

c) Các mô hình kiến trúc triển khai thực tiễn

Dựa trên cách tổ chức hệ thống tính toán, quá trình triển khai thuật toán OT được chia thành hai mô hình kiến trúc chính. Mô hình OT phân tán yêu cầu các máy khách kết nối ngang hàng và phải tự xử lý toàn bộ ma trận biến đổi. Phương pháp này đòi hỏi mỗi thiết bị phải duy trì một biểu đồ trạng thái toàn cục, dẫn đến độ phức tạp thuật toán cực cao và khó kiểm soát lỗi đồng bộ.

Ngược lại, mô hình OT tập trung thiết lập một máy chủ trung tâm đóng vai trò là nút điều phối luồng xử lý. Máy chủ này chịu trách nhiệm tiếp nhận toàn bộ các thao tác gửi lên, xếp tự theo thời gian, thực thi hàm biến đổi và phát tín hiệu chứa thao tác đã qua xử lý ngược lại cho toàn bộ mạng lưới máy khách. Kiến trúc này giúp kiểm soát vòng đời sự kiện một cách chặt chẽ, loại bỏ tình trạng tranh chấp tính toán và giảm tải đáng kể tài nguyên phần cứng cho các thiết bị của người dùng cuối.

2.2.2. Thuật toán CRDT

Khác biệt hoàn toàn với tư duy tính toán và biến đổi hành động của thuật toán OT, giải thuật CRDT tiếp cận bài toán đồng bộ bằng cách thiết kế lại toàn bộ nền tảng lưu trữ dữ liệu. Giải thuật này giải quyết xung đột ngay từ cấu trúc dữ liệu thay vì chờ đến khâu xử lý luồng sự kiện. Để làm rõ cơ chế vận hành, cấu trúc CRDT được phân tích qua ba khía cạnh kỹ thuật sau:

a) Cơ sở toán học và đặc tính hội tụ

CRDT là tập hợp các cấu trúc dữ liệu phân tán có khả năng tự động hội tụ trạng thái mà không cần cơ chế phân xử trung tâm. Nền tảng của hệ thống này được xây dựng dựa trên ba tính chất toán học cơ bản bao gồm tính giao hoán Commutativity, tính kết hợp Associativity và tính lũy đẳng Idempotency.

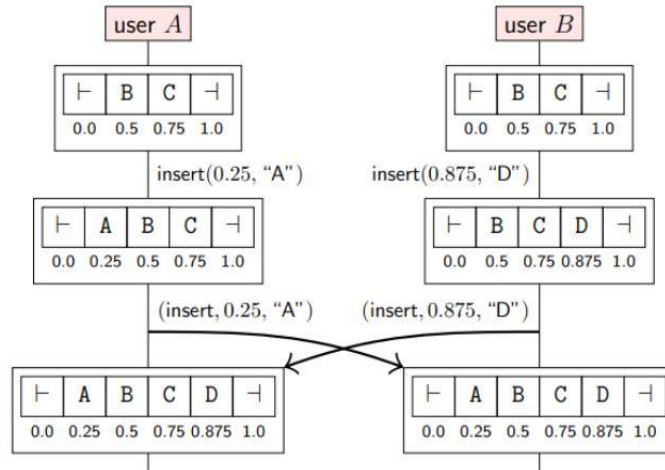
Nhờ các thuộc tính toán học chặt chẽ này, các nút mạng có thể tiếp nhận và áp dụng các thao tác cập nhật theo bất kỳ trình tự thời gian nào. Khi luồng dữ liệu truyền qua mạng bị xáo trộn do độ trễ, hệ thống không cần thực thi hàm biến đổi thao tác. Miễn là tất cả các máy khách trong mạng lưới đều nhận được cùng một tập hợp thông điệp, trạng thái dữ liệu cuối cùng trên toàn hệ thống chắc chắn sẽ hoàn toàn đồng nhất.

b) Cấu trúc mảng tuần tự và định danh phân số

Để áp dụng CRDT vào bài toán soạn thảo văn bản, hệ thống sử dụng cấu trúc mảng tuần tự. Trong cấu trúc mảng thông thường, các ký tự được quản lý bằng số nguyên tuyến tính. Tuy nhiên, CRDT loại bỏ cách đánh số này và cấp phát cho mỗi ký tự một định danh vị trí duy nhất, phân cấp và tuyệt đối không thể thay đổi toàn cục.

Các hệ thống CRDT thường sử dụng cơ chế nội suy định danh phân số hoặc cấu trúc cây đa phân. Khi một người dùng chèn ký tự mới vào giữa hai ký tự hiện có, hệ thống sẽ tự động phát sinh một định danh mới có giá trị nằm giữa hai định danh cũ. Do vị trí tuyệt đối của mọi ký tự là cố định, thao tác chèn hoặc xóa từ nhiều Client khác nhau không bao giờ làm dịch chuyển chỉ số của các vùng văn bản lân cận, loại trừ hoàn toàn hiện tượng xung đột không gian.

Text editing CRDT



Hình 2.5: Minh họa cơ chế cấp phát định danh phân số khi chèn ký tự mới trong cấu trúc dữ liệu CRDT.

c) Đặc tính kỹ thuật và giới hạn thực tiễn

Ưu điểm vượt trội của cấu trúc CRDT là khả năng tự hội tụ mạnh mẽ và độ phức tạp thuật toán thấp. Hệ thống loại bỏ hoàn toàn nhu cầu duy trì biểu đồ lịch sử thao tác hoặc thiết lập ma trận biến đổi phức tạp, biến giải thuật này trở thành nền tảng lý tưởng cho các kiến trúc mạng ngang hàng hoạt động ngoại tuyến.

Tuy nhiên, giới hạn cốt tử của CRDT nằm ở chi phí không gian lưu trữ và băng thông truyền tải mạng. Để duy trì tính duy nhất toàn cục, mỗi ký tự đơn lẻ phải đính kèm một chuỗi định danh siêu dữ liệu metadata cực kỳ dài và phức tạp. Đối với một văn bản lớn, dung lượng bộ nhớ cấp phát cho cấu trúc CRDT có thể phình to gấp hàng chục lần so với kích thước nội dung văn bản thực tế. Trong bối cảnh hệ thống kiến trúc Client-Server yêu cầu tối ưu hóa tốc độ gói tin qua WebSockets và tiết kiệm tài nguyên lưu trữ trên cơ sở dữ liệu MongoDB, mức độ tiêu hao bộ nhớ này là một nhược điểm lớn.

2.2.3. So sánh và định hướng lựa chọn giải thuật

Cả hai họ giải thuật OT và CRDT đều cung cấp nền tảng toán học vững chắc để giải quyết bài toán đồng bộ phân tán và đảm bảo tính nhất quán cuối cùng. Tuy nhiên, mỗi

phương pháp lại mang những đặc tính kỹ thuật trái ngược nhau, đòi hỏi quá trình đánh giá kỹ lưỡng dựa trên yêu cầu thực tiễn của kiến trúc phần mềm đang được xây dựng. Việc so sánh được thực hiện trên bốn tiêu chí cốt lõi bao gồm độ phức tạp giải thuật, tiêu hao băng thông mạng, dung lượng lưu trữ bộ nhớ và khả năng tích hợp hạ tầng.

Bảng 2.1: Bảng so sánh đặc tính kỹ thuật giữa giải thuật OT và CRDT.

Tiêu chí đánh giá	OT	CRDT
Độ phức tạp thuật toán	Rất cao. Đòi hỏi xây dựng ma trận biến đổi trạng thái phức tạp tại máy chủ trung tâm.	Thấp. Dựa trên toán học hội tụ tự nhiên, không cần xử lý hàm biến đổi trung gian.
Dung lượng lưu trữ	Tối ưu. Cấu trúc văn bản được lưu trữ dưới dạng chuỗi nguyên thủy, tiết kiệm không gian.	Rất lớn. Mỗi ký tự đi kèm bộ định danh siêu dữ liệu phức tạp, làm phình to bộ nhớ.
Băng thông mạng	Rất thấp. Gói tin truyền tải chỉ chứa thông số vị trí chỉ số và hành động cơ bản	Cao. Gói tin phải đính kèm toàn bộ chuỗi định danh phân cấp tính của ký tự.
Kiến trúc phù hợp	Tối ưu hóa cho cấu trúc Client-Server với máy chủ điều phối định tuyến.	Tối ưu hóa cho cấu trúc mạng ngang hàng và hoạt động ngoại tuyến.

Từ những đặc tính kỹ thuật được so sánh tại Bảng 2.1, giải thuật OT tập trung được lựa chọn làm phương pháp đồng bộ cốt lõi cho hệ thống. Sự lựa chọn này được đưa ra dựa trên ba yếu tố thực tiễn, bám sát với định hướng thiết kế ban đầu.

Thứ nhất, hệ thống được định hướng thiết kế theo kiến trúc Khách-Chủ. Mô hình OT tập trung hoàn toàn tương thích với kiến trúc này, cho phép chuyển giao toàn bộ gánh nặng tính toán ma trận biến đổi về phía máy chủ. Cơ chế này giúp các trình duyệt của người dùng duy trì hiệu năng mượt mà, không bị hao tổn tài nguyên CPU ngay cả khi chạy trên

các thiết bị có cấu hình phần cứng thấp.

Thứ hai, việc tối ưu hóa băng thông truyền tải qua giao thức WebSockets là ưu tiên hàng đầu để đạt được độ trễ thấp nhất. Giải thuật OT cho phép đóng gói các sự kiện gõ phím thành những thông điệp mạng cực kỳ nhỏ gọn, giảm thiểu tối đa rủi ro nghẽn cổ chai mạng so với việc phải truyền tải liên tục các cấu trúc siêu dữ liệu khổng lồ của CRDT.

Cuối cùng, ở góc độ cấp phát tài nguyên lưu trữ, việc sử dụng hệ quản trị cơ sở dữ liệu MongoDB và bộ nhớ đệm Redis yêu cầu tối ưu hóa không gian. Cấu trúc chuỗi ký tự nguyên thủy của giải thuật OT giúp tiết kiệm triệt để tài nguyên bộ nhớ đệm và dung lượng đĩa cứng, đảm bảo hệ thống duy trì khả năng mở rộng ngang khi số lượng phòng soạn thảo và lịch sử thao tác tăng vọt trong quá trình vận hành thực tế.

2.3. Cơ chế kiểm soát quan hệ nhân quả

Trong một hệ thống máy chủ đơn lẻ, việc xác định thứ tự trước sau của các thao tác rất đơn giản nhờ vào đồng hồ vật lý của hệ điều hành. Tuy nhiên, trong hệ thống phân tán mạng diện rộng, độ trễ truyền tải luôn biến động khiến đồng hồ vật lý giữa các máy khách và máy chủ không bao giờ đồng bộ tuyệt đối. Để giải thuật OT có thể tính toán chính xác thao tác nào cần được biến đổi, hệ thống bắt buộc phải thiết lập một cơ sở logic toán học để kiểm soát thời gian, gọi là cơ chế kiểm soát quan hệ nhân quả.

2.3.1. Quan hệ nhân quả *Happened-Before*

Nền tảng của việc kiểm soát thời gian trong hệ thống phân tán được xây dựng dựa trên lý thuyết Happened-Before do nhà khoa học Leslie Lamport định nghĩa. Lý thuyết này thiết lập một trật tự cục bộ thay vì cố gắng đồng bộ thời gian toàn cục. Theo đó, hệ thống chỉ quan tâm đến thứ tự logic của các sự kiện dựa trên luồng truyền tải thông điệp.

Quan hệ Happened-Before quy định ba trường hợp cơ bản đối với hai thao tác A và B bất kỳ trong mạng. Thứ nhất, nếu A và B xảy ra tuần tự trên cùng một máy khách và A xảy ra trước, thì A có quan hệ nhân quả dẫn đến B. Thứ hai, nếu thao tác A là sự kiện gửi gói tin từ máy khách và B là sự kiện nhận gói tin đó tại máy chủ, thì A chắc chắn xảy ra trước B. Cuối cùng và quan trọng nhất, nếu thao tác A và B được tạo ra tại hai trình duyệt

khác nhau ở cùng một thời điểm logic mà không bên nào biết về sự tồn tại của bên kia, thì A và B được gọi là hai sự kiện đồng thời.

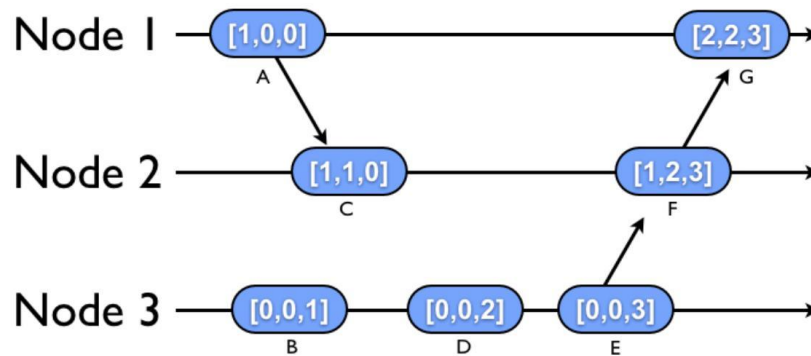
Việc xác định các sự kiện đồng thời là nhiệm vụ cốt lõi của máy chủ trung tâm. Nếu một thao tác đến sau nhưng có tính kế thừa logic từ thao tác trước, máy chủ chỉ việc áp dụng thẳng vào cơ sở dữ liệu. Ngược lại, nếu máy chủ phát hiện hai thao tác là đồng thời, hệ thống sẽ ngay lập tức kích hoạt luồng xử lý OT để phân giải xung đột không gian trước khi lưu trữ.

2.3.2. Thuật toán *Vector Clock*

Để hiện thực hóa lý thuyết Happened-Before vào mã nguồn phần mềm, hệ thống sử dụng cấu trúc dữ liệu Vector Clock. Đây là một thuật toán đánh dấu phiên bản logic, cho phép các nút mạng theo dõi và so sánh chính xác trạng thái nhân quả của nhau mà không cần dựa vào thời gian thực.

Về mặt cấu trúc, Vector Clock là một mảng chứa các bộ đếm số nguyên, trong đó mỗi phần tử đại diện cho trạng thái phiên bản của một nút mạng cụ thể. Mỗi khi một trình duyệt thực hiện thao tác gõ phím, nó sẽ tăng giá trị bộ đếm cục bộ của mình lên một đơn vị và đính kèm toàn bộ mảng Vector Clock hiện tại vào gói tin gửi lên máy chủ. Khi máy chủ nhận được gói tin, nó sẽ đối chiếu Vector Clock của trình duyệt với Vector Clock lưu trữ trên cơ sở dữ liệu trung tâm.

Quá trình so sánh toán học giữa hai mảng Vector Clock giúp máy chủ đưa ra quyết định chính xác tuyệt đối. Nếu mảng của trình duyệt nhỏ hơn hoặc bằng mảng của máy chủ, điều đó chứng tỏ trình duyệt đang thao tác trên một phiên bản văn bản cũ và thao tác này bị lỗi thời logic. Máy chủ sẽ truy xuất lịch sử các thao tác đã diễn ra trong khoảng chênh lệch phiên bản đó từ bộ nhớ đệm Redis, sau đó đưa vào hàm biến đổi OT để dịch chuyển tọa độ. Sau khi xử lý xong, máy chủ cập nhật lại Vector Clock tổng, lưu trữ trạng thái mới và phát tín hiệu đồng bộ về cho toàn bộ mạng lưới.



Hình 2.6: Minh họa cơ chế Vector Clock trong việc xác định trạng thái nhân quả giữa các sự kiện trong hệ thống phân tán.

2.4. Lý thuyết về truyền thông mạng và hạ tầng thời gian thực

Để các thuật toán đồng bộ và cơ chế kiểm soát nhân quả có thể vận hành hiệu quả, hệ thống yêu cầu một nền tảng hạ tầng mạng có độ trễ cực thấp và kiến trúc xử lý luồng dữ liệu linh hoạt. Phần này đi sâu phân tích các nguyên lý công nghệ cốt lõi được áp dụng để xây dựng kiến trúc tổng thể.

2.4.1. Giao thức truyền thông WebSockets

Trong các ứng dụng web truyền thống, giao thức HTTP là tiêu chuẩn giao tiếp mặc định. Tuy nhiên, HTTP hoạt động dựa trên cơ chế yêu cầu và phản hồi đơn hướng. Để cập nhật trạng thái theo thời gian thực bằng HTTP, máy khách phải liên tục gửi các yêu cầu kiểm tra trạng thái đến máy chủ. Phương pháp này tạo ra độ trễ lớn, lãng phí tài nguyên băng thông và làm quá tải máy chủ do phải liên tục thiết lập lại các kết nối mạng.

Để khắc phục rào cản này, giao thức WebSockets được thiết kế để cung cấp một kênh truyền thông song hướng liên tục qua một kết nối duy nhất. Sau chu trình bắt tay khởi tạo, kết nối mạng giữa trình duyệt và máy chủ luôn được giữ mở. Khi một người dùng gõ phím, gói tin chứa thao tác sửa đổi được đẩy trực tiếp qua luồng stream mạng đang mở mà không cần đính kèm các phần đầu tiêu đề công kênh như HTTP. Điều này giúp giảm thiểu tối đa băng thông tiêu thụ và rút ngắn thời gian phản hồi mạng xuống mức mili-giây, đáp ứng tiêu chuẩn khắt khe của hệ thống cộng tác.

2.4.2. Kiến trúc hướng sự kiện và Message Broker

Khi số lượng người dùng đồng thời tăng cao, việc máy chủ API tự mình xử lý trực tiếp mọi tính toán OT và ghi cơ sở dữ liệu sẽ dẫn đến tình trạng nghẽn cổ chai tài nguyên CPU. Kiến trúc hướng sự kiện được áp dụng để phân tách các thành phần phần mềm. Trong kiến trúc này, các thao tác của người dùng không được xử lý ngay mà được đóng gói thành các sự kiện và đẩy vào một hàng đợi trung gian.

Thành phần trung gian này được gọi là Message Broker. Vai trò cốt lõi của Message Broker là tiếp nhận, lưu trữ tạm thời và định tuyến các luồng sự kiện bất đồng bộ đến các tiến trình xử lý thích hợp. Đối với bài toán soạn thảo cộng tác, Message Broker đảm nhận nhiệm vụ cực kỳ quan trọng là duy trì tính tuần tự của luồng sự kiện. Mọi thao tác gõ phím thuộc về cùng một tài liệu bắt buộc phải được định tuyến vào chung một hàng đợi duy nhất để đảm bảo các tiến trình Worker thực thi mà trận biến đổi OT theo đúng trình tự thời gian logic, ngăn chặn hoàn toàn nguy cơ xáo trộn dữ liệu.

Hình 2.7: Kiến trúc hướng sự kiện sử dụng Message Broker để xếp hàng và định tuyến luồng thông điệp.

2.4.3. Cơ chế bộ nhớ đệm tốc độ cao

Trong quá trình thực thi thuật toán OT, máy chủ liên tục phải truy xuất lại lịch sử các thao tác cũ để so sánh và tính toán độ lệch chỉ số. Việc đọc và ghi dữ liệu liên tục vào ổ cứng cơ học hoặc ổ cứng trạng thái rắn sẽ tạo ra độ trễ giới hạn vật lý, không thể đáp ứng tốc độ xử lý thời gian thực.

Giải pháp tối ưu là sử dụng cơ chế lưu trữ trực tiếp trên bộ nhớ truy cập ngẫu nhiên, hay còn gọi là In-memory Caching. Dữ liệu lịch sử thao tác của các tài liệu đang trong phiên làm việc sôi động sẽ được chuyển hoàn toàn lên RAM. Nhờ tốc độ đọc ghi vượt trội của bộ nhớ chính, các tiến trình Worker có thể truy xuất chuỗi sự kiện và thực thi hàm tính toán toán học trong thời gian tính bằng micro-giây, tối đa hóa thông lượng xử lý của hệ thống.

2.4.4. Hệ quản trị cơ sở dữ liệu phi quan hệ

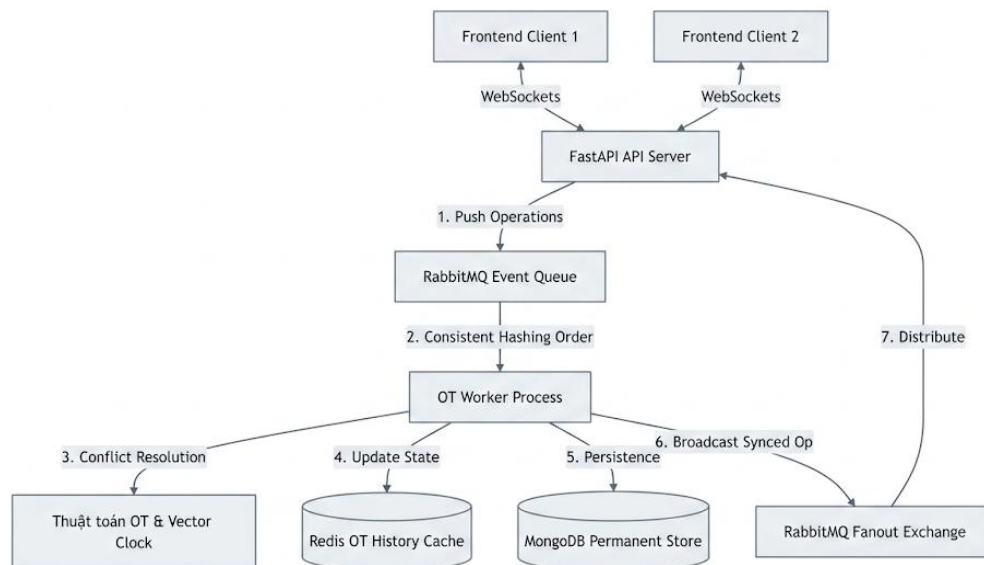
Lưu trữ lâu dài dữ liệu văn bản là khâu cuối cùng trong luồng xử lý. Các hệ quản trị cơ sở dữ liệu quan hệ truyền thống yêu cầu thiết lập các bảng có cấu trúc tĩnh khắt khe, không phù hợp với đặc thù dữ liệu thay đổi linh hoạt của ứng dụng soạn thảo.

Hệ quản trị cơ sở dữ liệu phi quan hệ NoSQL dạng tài liệu cung cấp kiến trúc lưu trữ mềm dẻo hơn. Dữ liệu được lưu trữ dưới định dạng các cấu trúc đối tượng độc lập, cho phép linh hoạt biểu diễn các trường thông tin không đồng nhất như siêu dữ liệu tài liệu, danh sách phân quyền, và nhật ký các thao tác chèn xóa phức tạp. Khả năng mở rộng ngang tự nhiên của kiến trúc NoSQL cũng giúp hệ thống dễ dàng phân mảnh dữ liệu trên nhiều cụm máy chủ vật lý, đảm bảo tính sẵn sàng cao khi lượng lưu trữ ngày càng phình to.

CHƯƠNG 3. TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG SOẠN THẢO VĂN BẢN CỘNG TÁC

3.1. Kiến trúc hệ thống

Kiến trúc tổng thể của hệ thống soạn thảo cộng tác được thiết kế theo mô hình hướng sự kiện phân tán. Trọng tâm của thiết kế này là việc tách biệt hoàn toàn tầng giao tiếp mạng và tầng xử lý tính toán logic, nhằm đảm bảo khả năng mở rộng linh hoạt và duy trì độ trễ thấp khi có nhiều kết nối đồng thời.



Hình 3.1: Sơ đồ kiến trúc tổng thể và luồng xử lý sự kiện đồng bộ.

3.1.1. Các thành phần hạ tầng cốt lõi

Dựa trên cấu trúc sơ đồ, hệ thống được cấu thành từ các nút dịch vụ độc lập, thực hiện các chức năng chuyên biệt:

- **Frontend Client**: Giao diện trình duyệt của người dùng, đóng vai trò tiếp nhận thao tác gõ phím và hiển thị kết quả. Giao tiếp với hạ tầng phía sau duy nhất thông qua giao thức WebSockets để đảm bảo tính thời gian thực.
- **FastAPI API Server**: Đóng vai trò là cổng API trung tâm. Dịch vụ này duy trì các

đường hầm WebSockets với người dùng nhưng không thực hiện bất kỳ tính toán toán học nào. Nhiệm vụ chính của nó là định tuyến luồng sự kiện vào và ra khỏi hệ thống mạng.

- RabbitMQ Event Queue: Hàng đợi thông điệp chịu trách nhiệm hấp thụ tải lưu lượng từ máy chủ API. Hạ tầng này đảm bảo các thao tác gõ phím không bị mất mát khi lưu lượng tăng đột biến và kiểm soát thứ tự phân phối cho cụm xử lý.
- OT Worker Process: Tiến trình xử lý nền độc lập, là trung tâm tính toán của hệ thống. Tiến trình này bóc tách dữ liệu từ hàng đợi và thực thi các mô-đun logic phức tạp.
- Thuật toán OT & Vector Clock: Khối logic nội bộ nằm trong Worker, chịu trách nhiệm xử lý ma trận Operational Transformation và đối chiếu đồng hồ logic để phân giải xung đột vị trí ký tự.
- Redis OT History Cache: Bộ nhớ đệm tốc độ cao, cung cấp dữ liệu tức thời về trạng thái văn bản và lịch sử thao tác để Worker thực thi thuật toán mà không bị nghẽn tốc độ đọc ghi.
- MongoDB Permanent Store: Hệ quản trị cơ sở dữ liệu phi quan hệ, đóng vai trò lưu trữ vĩnh viễn cấu trúc văn bản sau khi đã hội tụ thành công.
- RabbitMQ Fanout Exchange: Bộ định tuyến phát sóng nội bộ của RabbitMQ, có nhiệm vụ nhân bản gói tin kết quả và phân phối đồng loạt cho các máy chủ API.

3.1.2. Luồng hoạt động và xử lý sự kiện

Mọi thao tác chỉnh sửa văn bản từ phía người dùng đều trải qua một vòng đời khép kín và nghiêm ngặt, đảm bảo tính tuần tự và toàn vẹn dữ liệu toàn cục. Quá trình đồng bộ được kích hoạt ngay khi người dùng gõ phím tại giao diện máy khách. Sự kiện tương tác này lập tức được hệ thống đóng gói và truyền tải qua giao thức WebSockets đến cổng giao tiếp FastAPI. Nhằm tối ưu hóa hiệu năng, máy chủ API không trực tiếp xử lý tính toán mà chỉ đóng vai trò trạm trung chuyển, xuất bản gói tin vào hệ thống hàng đợi RabbitMQ rồi nhanh chóng giải phóng luồng mạng để đón nhận các kết nối mới.

Bên trong hạ tầng định tuyến, để ngăn chặn rủi ro xáo trộn thời gian của hệ thống phân tán, RabbitMQ áp dụng cơ chế bấm nhất quán dựa trên mã định danh của tài liệu. Kỹ thuật này cam kết mọi thao tác thuộc về cùng một phiên làm việc luôn được luân chuyển vào đúng một hàng đợi duy nhất. Từ hàng đợi này, dữ liệu được cấp phát cho một tiến trình OT Worker xử lý theo nguyên tắc vào trước ra trước. Ngay khi tiếp nhận thông điệp, Worker bắt đầu giai đoạn phân giải xung đột bằng cách tải mảng lịch sử từ bộ nhớ đệm Redis để kiểm tra chênh lệch phiên bản. Trong trường hợp phát hiện gói tin đến trễ hoặc có hiện tượng gõ phím đồng thời, khối logic Operational Transformation kết hợp Vector Clock sẽ can thiệp tính toán, dịch chuyển chỉ số tọa độ chèn hoặc xóa ký tự để bảo đảm tính đúng đắn cho cấu trúc văn bản.

Khi tọa độ chuẩn xác cuối cùng đã được xác lập, hệ thống bước vào giai đoạn cập nhật trạng thái. Tiến trình Worker thực hiện thao tác ghi đè tốc độ cao vào Redis nhằm làm mới mảng lịch sử đối chiếu, song song với việc lưu trữ vĩnh viễn trạng thái văn bản hội tụ xuống hệ quản trị MongoDB để phòng tránh rủi ro mất mát phần cứng. Khép lại chu trình xử lý, Worker xuất bản thao tác thành phẩm vào RabbitMQ Fanout Exchange. Bộ trao đổi này làm nhiệm vụ nhân bản và phân phối tự động gói tin đến toàn bộ các máy chủ FastAPI đang đăng ký lắng nghe. Ở chặng cuối, FastAPI tận dụng các đường hầm WebSockets hiện hữu để đẩy luồng dữ liệu cập nhật về lại màn hình của tất cả cộng tác viên, hoàn tất quá trình tự động vẽ lại giao diện với độ trễ ở mức tối thiểu.

3.2. Triển khai backend và thuật toán OT

3.2.1. Tầng giao tiếp thời gian thực và cổng điều phối API

Cổng giao tiếp API trung tâm vận hành dựa trên cơ chế bất đồng bộ nhằm tối ưu hóa tài nguyên mạng và duy trì kết nối hai chiều liên tục với các máy khách thông qua giao thức truyền thông đặc biệt.

Quản lý phiên làm việc và phòng tài liệu: Mỗi khi người dùng truy cập một văn bản, máy chủ thiết lập một đường hầm truyền thông riêng biệt được định danh bởi cặp mã số duy nhất đại diện cho tài liệu và người dùng. Hệ thống tự động phân nhóm các kết nối này

vào từng không gian bộ nhớ ảo tương ứng với từng phòng soạn thảo. Khi một thành viên mới tham gia, hệ thống thực hiện hai tác vụ đồng thời: một mặt, nó kiểm tra trạng thái trong bộ nhớ đệm hoặc cơ sở dữ liệu để lấy về cấu trúc đồng hồ logic hiện thời nhằm đồng bộ hóa trạng thái ban đầu cho máy khách ; mặt khác, nó phát tín hiệu quảng bá thông báo cho tất cả thành viên khác trong cùng không gian để cập nhật danh sách trực tuyến.

Phân tách và định tuyến luồng thông điệp: Tầng giao tiếp này thực hiện nguyên lý tách biệt trách nhiệm một cách triệt để. Các tương tác nhẹ, mang tính chất giao diện thuần túy như sự kiện di chuyển con trỏ chuột của người dùng, sẽ được máy chủ tiếp nhận và phát sóng trực tiếp ngay lập tức tới tất cả các thành viên trong cùng phòng soạn thảo mà không đẩy sâu vào hệ thống lưu trữ bên dưới nhằm tối ưu hóa độ trễ hiển thị. Ngược lại, đối với các thông điệp làm đột biến nội dung văn bản các hành động chèn hoặc xóa ký tự, máy chủ đóng vai trò như một trạm trung chuyển. Nó đóng gói toàn bộ ngữ cảnh dữ liệu bao gồm nội dung sửa đổi, đồng hồ logic và chỉ số phiên bản toàn cục rồi đẩy trực tiếp vào trực hàng đợi thông điệp nội bộ. Luồng xử lý mạng sau đó được giải phóng ngay lập tức để đón nhận các sự kiện tiếp theo mà không bị chặn bởi các tiến trình tính toán toán học phức tạp.

Giải phóng và thu dọn tài nguyên: Khi một máy khách ngắt kết nối hoặc rời khỏi phòng soạn thảo, hệ thống cập nhật lại danh sách thành viên trực tuyến và thông báo cho các máy khách còn lại. Đặc biệt, nếu người dùng cuối cùng rời khỏi phòng, hệ thống sẽ kích hoạt một tiến trình nền đặc biệt để quét lại toàn bộ nội dung văn bản hoàn chỉnh trong bộ nhớ đệm và thực hiện một lệnh ghi vĩnh viễn xuống cơ sở dữ liệu chính nhằm giải phóng vùng nhớ đệm một cách an toàn.

3.2.2. Message Broker Layer (RabbitMQ)

Hạ tầng RabbitMQ đóng vai trò điều phối trung tâm, thiết lập một hành lang tuần tự hóa nghiêm ngặt nhằm duy trì tính nhất quán và trật tự dòng sự kiện trong môi trường phân tán có mật độ tương tranh cao.

Định tuyến dựa trên thuật toán băm nhất quán để phân bổ tải trọng tính toán từ hàng nghìn tài liệu hoạt động cùng lúc, hệ thống cấu hình bộ định tuyến trung tâm sử dụng kỹ

thuật băm nhất quán. Khóa định tuyến được tính toán bằng cách băm chuỗi định danh độc bản của tài liệu thành một mã Hex thông qua giải thuật mã hóa dữ liệu, sau đó thực hiện phép toán lấy số dư với tổng số lượng đường ống phân phối hiện có. Cơ chế này cam kết một đặc tính kỹ thuật quan trọng, mọi thao tác chỉnh sửa, dù phát sinh từ nhiều người dùng khác nhau hay qua các cổng API vật lý khác nhau, nếu tác động lên cùng một văn bản thì bắt buộc phải xếp hàng vào chung một đường ống hàng đợi duy nhất.

Chuyển đổi tương tranh thành tuần tự nhằm loại bỏ Race Condition. Hệ thống áp dụng cấu hình giới hạn định mức phân phối nghiêm ngặt cho các hàng đợi, cưỡng ép mỗi đường ống dữ liệu tại một thời điểm chỉ cấp phát duy nhất một thông điệp cho một tiến trình xử lý nền tiêu thụ. Gói tin tiếp theo chỉ được phân phối khi gói tin trước đó đã hoàn tất chu trình tính toán và xác nhận xử lý thành công. Chiến lược vào trước - ra trước FIFO này thiết lập một dòng thời gian tuyến tính tuyệt đối cho từng tài liệu, loại bỏ hoàn toàn các lỗi xung đột ghi đè do tính toán song song, tạo nền tảng vững chắc cho việc thực thi thuật toán phân giải xung đột hình học.

3.2.3. Tầng tiêu thụ và xử lý sự kiện

Thay vì tích hợp trực tiếp toàn bộ logic tính toán vào cổng giao tiếp thời gian thực WebSockets, hệ thống thiết lập một tiến trình xử lý nền hoạt động độc lập đóng vai trò là bộ não chuyên trách tính toán thuật toán đồng bộ. Việc phân rã kiến trúc này giúp giải phóng tối đa tài nguyên cho cổng giao tiếp API, ngăn chặn nguy cơ quá tải hệ thống khi có lượng lớn người dùng cùng thao tác đồng thời.

Tiến trình xử lý nền vận hành theo một chu trình hoạt động khép kín với năm nhiệm vụ trọng tâm sau:

Tuần tự hóa luồng công việc: Tiến trình xử lý nền được cấu hình để chỉ rút và tiêu thụ từng thao tác một từ đường ống hàng đợi trung chuyển của trục thông điệp. Đặc tính kỹ thuật này là chìa khóa then chốt để giải quyết bài toán đồng bộ: nó cưỡng ép toàn bộ các thao tác chỉnh sửa dù xảy ra đồng thời ở phía các máy khách phải xếp hàng tuần tự để xử lý theo một đường thẳng tuyến tính trên máy chủ, loại bỏ hoàn toàn các tranh chấp bất đồng

bộ.

Bộ lọc phiên bản và loại trừ dị thường: Trước khi đưa vào ma trận tính toán, tiến trình đóng vai trò như một người gác cổng kiểm soát chất lượng dữ liệu. Hệ thống thực hiện đối chiếu chỉ số phiên bản đính kèm trong thao tác gửi đến với chỉ số phiên bản thực tế được lưu trữ bởi cơ sở dữ liệu. Nếu phát hiện thao tác được gửi lên từ một máy khách đã bị mất kết nối quá lâu, tiến trình sẽ từ chối thực thi và ngay lập tức phát đi một tín hiệu cưỡng ép khôi phục trạng thái, yêu cầu thiết bị đó phải ghi đè và làm mới lại toàn bộ nội dung văn bản mới nhất.

Thực thi thuật toán đồng bộ OT Pipeline: Đối với các thao tác hợp lệ và vượt qua vòng kiểm duyệt phiên bản, tiến trình tiến hành truy xuất mảng nhật ký lịch sử chỉnh sửa ngắn hạn từ bộ nhớ đệm tốc độ cao. Dữ liệu này sau đó được đưa vào buồng tính toán của thuật toán biến đổi tọa độ hình học chuỗi để hiệu chỉnh, tịnh tiến lại các chỉ mục chèn hoặc xóa ký tự sao cho khớp với ngữ cảnh thực tế của văn bản.

Điều phối lưu trữ đa tầng: Khi tọa độ chuẩn xác cuối cùng đã được xác lập, tiến trình hoạt động như một nhạc trưởng điều phối luồng lưu trữ: một mặt, nó lưu vết nguyên tử toàn bộ giao dịch vào cơ sở dữ liệu vật lý dài hạn để làm nhật ký kiểm toán; mặt khác, nó ghi đè và cập nhật ngay lập tức bản nháp toàn văn mới lên bộ nhớ đệm RAM tốc độ cao. Tính chất nguyên tử được bảo toàn chặt chẽ ở bước này, đảm bảo dữ liệu chỉ được công nhận khi cả hai tầng lưu trữ đều ghi nhận thành công.

Phát sóng kết quả hội tụ: Sau khi trạng thái mới đã được ghi nhận an toàn tại các tầng lưu trữ, tiến trình xử lý nền chủ động đẩy gói tin kết quả đã chuẩn hóa vào bộ định tuyến phát sóng nội bộ. Bộ định tuyến này chịu trách nhiệm nhân bản và lan truyền thông điệp đến tất cả các cổng WebSocket đang hoạt động để đẩy ngược lại giao diện hiển thị của tất cả cộng tác viên trong phòng soạn thảo theo thời gian thực.

3.2.4. Logic phân giải xung đột và ma trận thuật toán OT

Tiến trình xử lý nền hoạt động như một thực thể độc lập, liên tục tiêu thụ dữ liệu từ các hàng đợi và vận hành khối logic Operational Transformation kết hợp Đồng hồ Vector.

Luồng xử lý một thao tác mới được hiện thực hóa qua ba chặng logic nghiêm ngặt.

a. Cơ chế kiểm soát phiên bản và loại bỏ thao tác lỗi thời

Trước khi đưa vào ma trận tính toán, hệ thống đối chiếu chỉ số phiên bản toàn cục đính kèm trong gói tin của máy khách với chỉ số phiên bản thực tế được lưu trữ bởi cơ sở dữ liệu trên máy chủ.

- Nếu chỉ số phiên bản của máy khách nhỏ hơn phiên bản hiện thời trên hệ thống, điều này phản ánh một kịch bản mất đồng bộ nghiêm trọng.
- Hệ thống sẽ lập tức từ chối thực thi thao tác này. Đồng thời, máy chủ phát ra một thông điệp cưỡng ép quay lui trạng thái, gửi toàn bộ nội dung văn bản chuẩn xác cùng chỉ số phiên bản toàn cục mới nhất về để ép máy khách phải ghi đè và làm mới lại hoàn toàn giao diện cục bộ trước khi được phép gõ phím trở lại.

b. Bộ lọc quan hệ nhân quả bằng đồng hồ vector

Đối với các thao tác vượt qua vòng kiểm tra phiên bản, hệ thống tiến hành truy xuất mảng nhật ký lịch sử các thao tác đã được xác thực trước đó của tài liệu từ bộ nhớ đệm tốc độ cao. Hệ thống duyệt mảng lịch sử này theo trình tự thời gian tăng dần và áp dụng một bộ lọc toán học dựa trên cấu trúc đồng hồ Vector dưới dạng từ điển.

- Bộ lọc so sánh giá trị đồng hồ của từng thành viên lưu trong thao tác lịch sử với giá trị đồng hồ tương ứng của thành viên đó được ghi nhận trong thao tác mới gửi lên.
- Nếu giá trị đồng hồ trong lịch sử lớn hơn giá trị mà máy khách hiện tại biết khi sinh ra thao tác, hệ thống xác định đây là một thao tác tương tranh song song. Thao tác mới bắt buộc phải đưa vào ma trận biến đổi hình học. Các thao tác không thỏa mãn điều kiện này sẽ được bỏ qua nhằm bảo toàn trọn vẹn quan hệ nhân quả Happened-Before.

c. Hiện thực ma trận biến đổi tọa độ hình học chuỗi

Hạt nhân xử lý toán học nhận vào hai thực thể thao tác tương tranh và thực hiện hiệu chỉnh chỉ mục vị trí ký tự dựa trên bốn kịch bản cốt lõi:

- Chèn tương tác với chèn: Nếu chỉ mục của thao tác mới nhỏ hơn thao tác trong lịch sử, tọa độ được giữ nguyên. Nếu lớn hơn, chỉ mục của thao tác mới được tịnh tiến về phía bên phải một khoảng bằng đúng độ dài chuỗi ký tự của thao tác lịch sử. Trong tình huống đặc biệt khi hai chỉ mục hoàn toàn trùng nhau, hệ thống kích hoạt cơ chế phân xử tối cao dựa trên việc so sánh chuỗi định danh người dùng theo thứ tự từ điển. Thao tác của người dùng có mã định danh nhỏ hơn được ưu tiên giữ nguyên vị trí, trong khi thao tác của người dùng còn lại bắt buộc phải dời sang phải, nhường không gian hiển thị cho dữ liệu đến trước.
- Chèn tương tác với xóa: Nếu điểm chèn nằm phía trước vùng bị xóa, chỉ mục được giữ nguyên. Nếu điểm chèn nằm sau vùng bị xóa, chỉ mục được dịch lùi về phía bên trái một khoảng bằng độ dài chuỗi văn bản bị xóa. Trường hợp điểm chèn nằm lọt vào chính giữa khoảng ký tự đang bị xóa của thao tác trước, hệ thống xử lý hiện tượng nuốt chữ bằng cách cưỡng ép dời chỉ mục chèn trượt về mép bên trái của vùng bị xóa để bảo toàn ý định gõ phím của người dùng.
- Xóa tương tác với chèn: Nếu vùng xóa nằm hoàn toàn trước điểm chèn, tọa độ được giữ nguyên. Nếu vùng xóa bắt đầu từ điểm chèn trở về sau, chỉ mục bắt đầu xóa được tịnh tiến sang phải một khoảng bằng độ dài chuỗi được chèn. Nếu điểm chèn mới nằm lọt vào giữa khoảng ký tự đang bị xóa của thao tác cũ, hệ thống kích hoạt cơ chế cắt xén. Thao tác xóa ban đầu bị phân tách làm hai mảnh rời rạc: mảnh thứ nhất chịu trách nhiệm xóa phần văn bản từ điểm bắt đầu đến điểm chèn, mảnh thứ hai xóa phần còn lại bắt đầu từ vị trí chèn mới cộng thêm độ dài của ký tự vừa được chèn.
- Xóa tương tác với xóa: Nếu hai vùng xóa không giao nhau, hệ thống giữ nguyên tọa độ hoặc dịch lùi chỉ mục của thao tác nằm sau. Nếu xuất hiện vùng giao thoa trực tiếp, hệ thống áp dụng luật toán học tập hợp để xử lý hiện tượng Trùng lặp. Nó tính toán vùng trùng nhau và gọt bỏ phần ký tự đó ra khỏi nội dung của thao tác đến sau nhằm tránh lỗi xóa lặp lại, đồng thời lùi chỉ mục bắt đầu của thao tác sau một lượng tương ứng với số ký tự thực tế đã bị xóa phía trước nó. Nếu vùng xóa của thao tác

mới bị bao trùm hoàn toàn bởi thao tác cũ, nó được biến đổi thành một thao tác rỗng vô hại.

d. Sửa lỗi domino tọa độ và phát tán trạng thái

Do hệ thống xử lý thao tác chuỗi theo khối và có khả năng phân tách một hành động thành mảng nhiều hành động nhỏ sau quá trình cắt xén, việc thực thi mảng kết quả này lên chuỗi văn bản phẳng có thể làm thay đổi chiều dài tổng thể ngay trong vòng lặp. Để khắc phục triệt để hiệu ứng domino xô lệch tọa độ, hệ thống thực hiện một quy tắc bắt buộc tự động sắp xếp lại mảng các thao tác thành phẩm theo thứ tự chỉ mục vị trí giảm dần. Việc áp dụng các thao tác sửa đổi từ phải qua trái đảm bảo các hành động ở cuối văn bản không bao giờ làm ảnh hưởng hay lệch chỉ số của các hành động ở đầu văn bản.

Sau khi chuẩn hóa, nội dung mới được cập nhật trực tiếp vào chuỗi văn bản thô trên bộ nhớ RAM, tăng chỉ số đồng hồ logic toàn cục thêm một đơn vị và xuất bản thông điệp đồng bộ lên bộ định tuyến phát sóng nội bộ để đẩy về tất cả các máy khách thông qua các kênh WebSocket đang mở.

3.2.5. Tầng bộ nhớ đệm trung tâm

Để đạt được hiệu năng đáp ứng thời gian thực với độ trễ cực thấp ở mức micro-giây cho mỗi sự kiện gõ phím, hệ thống loại bỏ hoàn toàn các thao tác đọc và ghi đĩa liên tục xuống cơ sở dữ liệu vật lý trong suốt tiến trình người dùng tương tác. Thay vào đó, toàn bộ trạng thái vận hành tức thời được duy trì tập trung trên nền tảng Redis - hệ thống lưu trữ dữ liệu trực tiếp trên bộ nhớ RAM.

Trong mô hình kiến trúc này, Redis không chỉ đóng vai trò làm bộ nhớ đệm tăng tốc đơn thuần mà trực tiếp gánh vác năm chức năng cốt lõi, thiết lập một hành lang quản lý trạng thái đồng bộ toàn diện cho hệ thống phân tán:

a. Bộ đệm bản nháp văn bản

Đây là vùng lưu trữ đại diện cho trạng thái nội dung toàn văn hiện thời của tài liệu đang được cộng tác.

- Cơ chế hoạt động: Khi một phòng soạn thảo được kích hoạt, hệ thống sẽ kiểm tra bộ nhớ đệm; nếu xảy ra hiện tượng thiếu hụt dữ liệu đệm, nội dung từ cơ sở dữ liệu bền vững sẽ được nạp lên cấu trúc chuỗi phẳng của Redis. Mỗi khi khối xử lý toán học hoàn tất việc biến đổi tọa độ cho một thao tác mới, hệ thống sẽ thực hiện kỹ thuật cắt và ghép chuỗi trực tiếp trên bộ nhớ RAM để cập nhật bản nháp này, đồng thời ghi đè toàn bộ chuỗi ký tự mới lên bộ nhớ đệm.
- Vai trò: Giải pháp lưu trữ toàn văn trên RAM giúp hệ thống xử lý mượt mà hàng nghìn lượt đột biến văn bản mỗi giây từ nhiều nguồn đồng thời mà không tạo ra áp lực đọc ghi lên ổ đĩa cứng vật lý.

b. Bộ đệm nhật ký thao tác

Song song với chuỗi văn bản thô, hệ thống duy trì một cấu trúc dữ liệu dạng danh sách tuyến tính liên tục trên Redis để lưu giữ nhật ký các thao tác chỉnh sửa đã được máy chủ phê duyệt.

- Cơ chế hoạt động: Mỗi khi một hành động chèn hoặc xóa ký tự được xác nhận tính đúng đắn, gói tin mang theo nội dung sửa đổi, mã định danh người dùng và mảng đồng hồ logic sẽ được đẩy vào đầu danh sách đệm. Khi một người dùng mới thực hiện luồng tham gia vào phòng, hệ thống lập tức truy xuất phần tử mới nhất ở vị trí đầu danh sách này để lấy ra trạng thái đồng hồ logic hiện thời nhằm thiết lập cấu trúc đồng bộ ban đầu cho máy khách.
- Vai trò: Cấu trúc danh sách này cung cấp một cửa sổ lịch sử tốc độ cao, cho phép khối thuật toán biến đổi thao tác có thể thực hiện duyệt ngược và lọc quan hệ nhân quả một cách tức thời để phân giải xung đột, thay vì phải thực hiện các câu lệnh truy vấn quét bảng tổn kém trên cơ sở dữ liệu vật lý.

c. Quản lý danh sách người đóng góp

Để phục vụ cho công tác thống kê và định danh phiên bản lưu trữ, hệ thống triển khai cấu trúc tập hợp phần tử độc bản trên bộ nhớ đệm.

- Cơ chế hoạt động: Xuyên suốt chu kỳ chỉnh sửa của một tài liệu, ngay khi một tiến

trình xử lý nền áp dụng thành công các thao tác sửa đổi của một người dùng cụ thể vào bản nháp văn bản, mã định danh của thành viên đó sẽ được thêm vào tập hợp lưu trữ tương ứng với tài liệu đó trên Redis. Do đặc tính của cấu trúc tập hợp, các mã định danh trùng lặp sẽ tự động bị gọt bỏ.

- Vai trò: Chức năng này cho phép hệ thống ghi nhận chính xác và tức thời toàn bộ danh sách những thành viên thực sự có đóng góp nội dung trong chu kỳ hiện tại, làm cơ sở dữ liệu metadata để gắn vào cấu trúc tài liệu khi hệ thống thực hiện luồng sao lưu bền vững.

d. Bộ đếm tần suất thao tác

Mô hình sao lưu định lượng của hệ thống hoạt động dựa trên một cơ chế kích hoạt bằng số nguyên nguyên tử được duy trì trực tiếp trên RAM.

- Cơ chế hoạt động: Đối với mỗi phòng tài liệu, một biến đếm số lượng thao tác tích lũy sẽ được thiết lập trên Redis. Mỗi khi hệ thống áp dụng và cập nhật thành công một chuỗi văn bản mới vào bộ nhớ đệm, giá trị của biến đếm này sẽ tự động được tăng thêm một đơn vị thông qua các phép toán nguyên tử của Redis. Khi giá trị bộ đếm chạm ngưỡng giới hạn an toàn quy định cụ thể là mốc mười thao tác gõ phím liên tục, hệ thống sẽ nhận diện điều kiện biên và chủ động kích hoạt một luồng xử lý nền để xả toàn bộ dữ liệu văn bản hoàn chỉnh xuống cơ sở dữ liệu chính, sau đó tái thiết lập giá trị bộ đếm về mức không.
- Vai trò: Cơ chế này đảm bảo dữ liệu luôn có các điểm kiểm soát an toàn theo khối lượng tương tác mà không cần phải thực hiện ghi đĩa liên tục trên từng phím gõ.

e. Theo dõi độ trễ thời gian

Để nhận biết trạng thái tĩnh lặng của phòng soạn thảo khi người dùng tạm ngừng tương tác, hệ thống tận dụng cơ chế quản lý thời gian trên bộ nhớ đệm.

- Cơ chế hoạt động: Tại thời điểm một thao tác gõ phím được ghi nhận và đồng bộ hóa thành công vào chuỗi văn bản trên RAM, hệ thống đồng thời lấy nhãn thời gian hiện tại của máy chủ và ghi đè vào một khóa lưu trữ mốc thời gian sửa đổi gần nhất

trên Redis. Một tiến trình nền độc lập sẽ liên tục vận hành theo chu kỳ quét; nếu khoảng chênh lệch giữa thời gian hệ thống hiện thời và mốc thời gian sửa đổi cuối cùng vượt quá ngưỡng nhàn rỗi cấu hình (cụ thể là mốc mười giây), đồng thời bộ đếm thao tác tích lũy ghi nhận có dữ liệu chưa lưu, hệ thống sẽ xác định tài liệu đã rơi vào trạng thái tĩnh lặng hoàn toàn.

- Vai trò: Chỉ số theo dõi thời gian này là điều kiện tiên quyết để kích hoạt chiến lược sao lưu trì hoãn và dọn dẹp tài nguyên cuối kỳ, đảm bảo toàn vẹn dữ liệu văn bản kể cả khi xảy ra các sự cố mất nguồn phần cứng đột ngột.

3.2.6. Tầng lưu trữ bền vững & đối soát (Persistence Layer với MongoDB)

Trong một hệ thống đồng bộ thời gian thực, cơ sở dữ liệu vật lý không tham gia trực tiếp vào từng thao tác gõ phím của người dùng nhằm tránh tình trạng quá tải. Thay vào đó, hệ thống sử dụng MongoDB làm nền tảng lưu trữ, đối soát và quản lý lịch sử dài hạn.

Để tối ưu hóa hiệu năng truy vấn, dữ liệu trong MongoDB được tổ chức thành 4 tập hợp cốt lõi:

a. Bảng quản lý người dùng

- Mục đích: Quản lý thông tin định danh.
- Cách thức lưu trữ: Lưu trữ thông tin tài khoản, mật khẩu, và hồ sơ người dùng. Dữ liệu này được đánh chỉ mục kỹ lưỡng để phục vụ quá trình xác thực cực nhanh mỗi khi người dùng kết nối vào hệ thống WebSocket.

b. Bảng tài liệu trung tâm (Documents Collection)

- Mục đích: Hoạt động như tham chiếu chuẩn mực cho toàn bộ hệ thống phân tán tại thời điểm hiện tại.
- Cách thức lưu trữ: Mỗi một văn bản chỉ có duy nhất một bản ghi tại đây. Bản ghi này chứa các thông tin:
 - Siêu dữ liệu: Tiêu đề, chủ sở hữu tài liệu.
 - Bản nháp: Chứa toàn bộ nội dung văn bản ở trạng thái mới nhất được chép từ

Redis sang.

- Bộ quản lý phiên bản: Lưu trữ số hiệu phiên bản vòng đời Epoch và Đồng hồ logic tổng Global Clock. Bất kỳ ai cố tình gửi thao tác mang số hiệu cũ hơn mốc này sẽ bị hệ thống từ chối để tránh làm hỏng dữ liệu.

c. Bảng lịch sử giao dịch

- Mục đích: Ghi nhận mọi hạt sạn nhỏ nhất trong quá trình chỉnh sửa.
- Cách thức lưu trữ: Đây là một chuỗi nhật ký khổng lồ có dạng thêm mới liên tục. Mỗi khi thuật toán OT tính toán xong một thao tác (chèn chữ, xóa chữ), thao tác đó được đóng gói thành một giao dịch và lưu vào đây kèm theo Dấu thời gian máy chủ.
- Vai trò: Nhật ký này đóng vai trò như chiếc "Hộp đen". Nếu Redis bị mất dữ liệu, hoặc thuật toán phát hiện có nhiều người gõ đồng thời, nó sẽ quét bảng này để dò lại các bước chỉnh sửa trong quá khứ nhằm tính toán lại tọa độ biến đổi.

d. Bảng điểm sao lưu đóng gói

- Mục đích: Đóng băng trạng thái để dễ dàng phục hồi.
- Cách thức lưu trữ: Nếu bảng Lịch sử Giao dịch quá dài, việc đọc lại từ đầu để khôi phục văn bản sẽ làm sập hệ thống. Do đó, cứ sau một mốc thời gian hoặc đạt một lượng chỉnh sửa nhất định, hệ thống tạo ra một điểm Sao lưu tại bảng này.
- Chi tiết: Một Checkpoint sẽ "chụp ảnh" lại toàn bộ chuỗi văn bản tại giây phút đó, kèm theo danh sách những cá nhân đã tham gia đóng góp và chốt lại số hiệu phiên bản. Nhờ đó, người dùng có thể mở ra một giao diện "Lịch sử Phiên bản" để xem lại hoặc quay ngược thời gian về một Checkpoint bất kỳ cực kỳ nhanh chóng.

3.3. Triển khai Frontend

Phần này trình bày các giải pháp kỹ thuật phía máy khách nhằm thiết lập trình soạn thảo trực quan, bắt và gom cụm các sự kiện gõ phím, thực thi thuật toán OT cục bộ để đồng bộ thời gian thực và quản lý các trạng thái giao diện cộng tác.

3.3.1. Kiến trúc Client và trình soạn thảo phi trạng thái

Để đảm bảo hiệu năng kết xuất hình ảnh và trải nghiệm tương tác trực quan cao, ứng dụng khách được triển khai bằng framework React 19 kết hợp với cấu trúc định tuyến hiện đại của Next.js.

a. Cấu trúc hiển thị thay thế vùng nhập liệu truyền thống

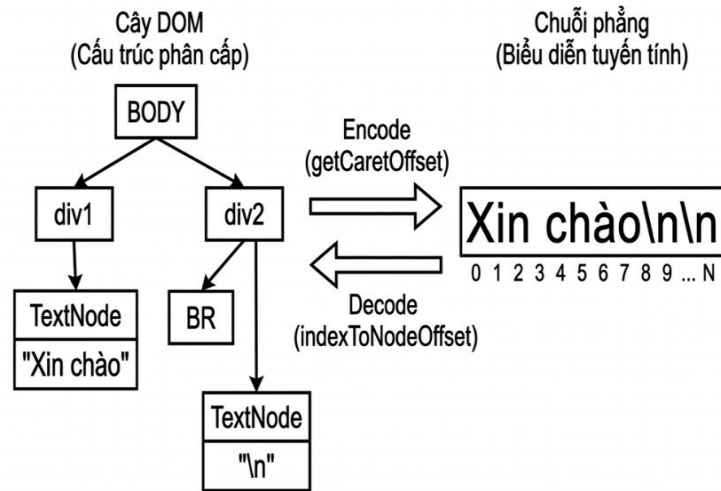
Hệ thống không sử dụng thẻ `<textarea>` truyền thống vì thẻ này giới hạn khả năng định dạng văn bản nâng cao và không hỗ trợ kết xuất nhiều định vị con trỏ chuột độc lập cùng lúc. Thay vào đó, hệ thống sử dụng một thẻ `div` được kích hoạt thuộc tính `contentEditable` bằng `true` của nền tảng HTML5. Trình soạn thảo hoạt động theo mô hình phi trạng thái, nội dung văn bản thuần túy được chuyển đổi và đồng nhất hóa thành cấu trúc DOM chuẩn hóa của trình duyệt.

Cụ thể, mỗi dòng văn bản được bọc trong một thẻ `<div>` phân dòng độc lập. Đối với dòng trống hoàn toàn, hệ thống biểu diễn bằng một thẻ `<div>` chứa một phân tử ngắt dòng `<br>` duy nhất bên trong. Điều này giúp giao diện duy trì khoảng cách dòng chính xác mà không làm phá vỡ logic tính toán.

b. Logic thuật toán ánh xạ chỉ mục con trỏ chuột

Thuật toán OT và hệ thống máy chủ chỉ làm việc trên một chuỗi văn bản phẳng duy nhất có chỉ số tuyến tính từ 0 đến N. Trong khi đó, trình duyệt lại quản lý vị trí con trỏ chuột theo cấu trúc cây DOM phức tạp gồm các nút văn bản và chỉ số độ lệch cục bộ trong từng nút. Máy khách giải quyết bài toán chuyển đổi hai chiều này bằng logic xử lý chuyên biệt dưới đây.

Cây DOM HTML5 vs Chuỗi phẳng



Hình 3.2: Lưu đồ thuật toán mã hóa cấu trúc hiển thị DOM sang chỉ mục logic phẳng.

- Logic hàm chuyển đổi DOM sang chỉ mục

Mục tiêu cốt lõi của hàm mã hóa là quy đổi vị trí trực quan của con trỏ trên giao diện thành một chỉ mục số nguyên duy nhất bên trong chuỗi dữ liệu phẳng của hệ thống. Để khởi chạy quá trình này, hệ thống tương tác trực tiếp với Selection API của trình duyệt nhằm trích xuất thông tin chi tiết về vùng chọn hiện tại. Thông qua giao diện lập trình này, máy khách xác định chính xác nút cấu trúc DOM đang chứa con trỏ cũng như khoảng độ lệch vật lý so với điểm bắt đầu của nút. Các thông số thu thập được sẽ đóng vai trò là tọa độ đích để thuật toán tiến hành đối chiếu trong tiến trình nội suy tiếp theo.

Sau khi xác định tọa độ đích, hệ thống khởi tạo một bộ duyệt để tiến hành quét toàn bộ cây DOM của trình soạn thảo bắt đầu từ phần tử gốc. Tiến trình duyệt cây được thực thi tuần tự từ trên xuống dưới và từ trái sang phải nhằm phản ánh chính xác luồng dữ liệu theo đúng thứ tự hiển thị trên màn hình. Trong quá trình quét không gian này, một biến đếm được sử dụng để liên tục tích lũy chiều dài của các nút văn bản mà bộ duyệt đi qua. Đặc biệt, hệ thống được thiết kế để nhận diện các ranh giới phân dòng hoặc thẻ ngắt dòng độc lập, từ đó tự động cộng thêm một đơn vị vào biến đếm để đại diện cho ký tự chuyển dòng ẩn bên trong chuỗi tuyến tính.

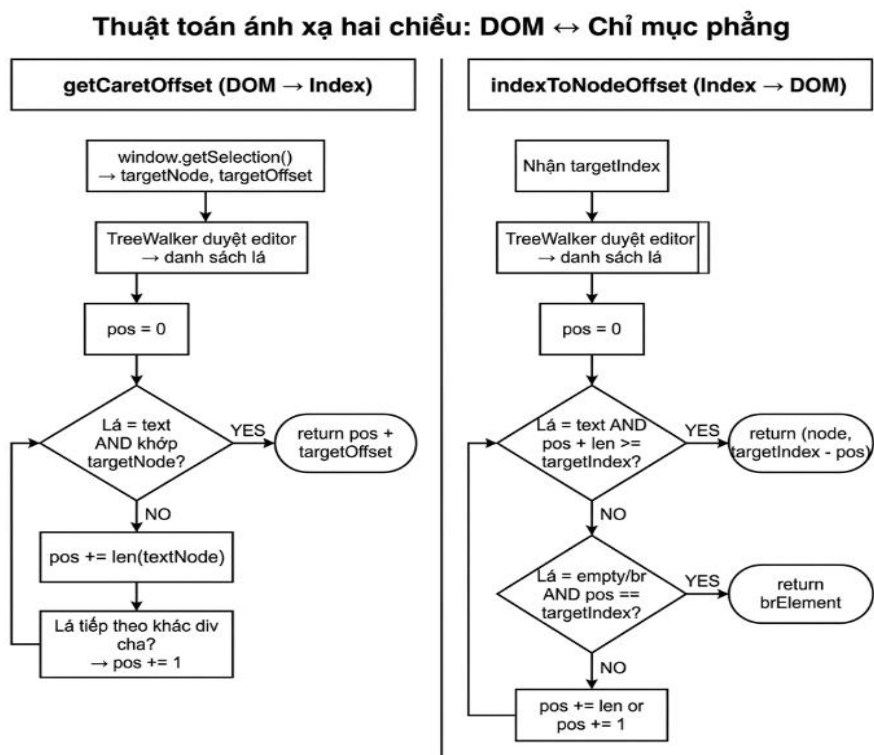
Vòng lặp duyệt cây DOM sẽ tự động kết thúc ngay khi hệ thống chạm tới đúng nút cấu trúc đích đã được xác định thông qua Selection API ở bước đầu. Giá trị tích lũy thu được tại khoảnh khắc dừng này chính là chỉ mục tuyến tính tuyệt đối của không gian soạn thảo. Con số chỉ mục này cung cấp tham số đầu vào thiết yếu để ứng dụng khởi tạo thao tác chỉnh sửa cục bộ, xuất bản gói tin đồng bộ lên máy chủ và thực thi chính xác thuật toán biến đổi không gian.

- Logic hàm chuyển đổi chỉ mục sang vị trí DOM

Hàm giải mã thực hiện nhiệm vụ đảo ngược nhằm định vị lại tọa độ vật lý của con trỏ chuột dựa trên tham số đầu vào là một chỉ mục số nguyên tuyến tính. Bài toán này phát sinh khi máy khách tiếp nhận thao tác chỉnh sửa từ máy chủ hoặc sau khi thuật toán đồng bộ vừa hiệu chỉnh lại vị trí con trỏ cục bộ. Để trình duyệt có thể vẽ lại con trỏ chính xác, hệ thống cần nội suy tọa độ phẳng này ngược trở lại vào bên trong cây cấu trúc phức tạp đang được kết xuất trên màn hình.

Để giải quyết vấn đề, thuật toán khởi tạo một bộ duyệt để quét toàn bộ cây cấu trúc của trình soạn thảo theo đúng thứ tự hiển thị từ trên xuống dưới. Trong quá trình quét không gian này, mỗi nút văn bản hoặc phần tử ngắt dòng đều được quy ước thành số lượng ký tự tương đương trong chuỗi phẳng. Hệ thống duy trì một biến đếm để liên tục cộng dồn tổng số ký tự đã đi qua, tạo thành thước đo hệ quy chiếu nhằm đối chiếu trực tiếp với chỉ mục đích cần tìm.

Khi giá trị tích lũy tiến đến hoặc vượt quá chỉ mục đích, hệ thống lập tức xác định được điểm rơi của con trỏ và dừng vòng lặp. Nếu tọa độ rơi vào giữa một đoạn văn bản, hệ thống tính toán độ lệch ký tự tương ứng để đặt con trỏ vào bên trong nút đó; ngược lại, nếu rơi vào ranh giới ngắt dòng, hệ thống ánh xạ trực tiếp vào phần tử phân dòng. Trong trường hợp chỉ mục đầu vào vượt quá tổng dung lượng hiện tại của tài liệu, thuật toán sẽ tự động neo con trỏ vào vị trí cuối cùng của không gian soạn thảo nhằm bảo đảm sự liên mạch cho trải nghiệm nhập liệu của người dùng.



Hình 3.3: Lưu đồ thuật toán hai chiều ánh xạ tọa độ giữa DOM và chỉ mục phẳng

3.3.2. Phát hiện thay đổi cục bộ và thuật toán đồng bộ máy khách

Trong kiến trúc phân tán thời gian thực, hiện tượng trễ mạng là không thể tránh khỏi. Nếu máy khách chờ đợi máy chủ xác nhận từng phím gõ rồi mới hiển thị lên màn hình, người dùng sẽ trải qua cảm giác khựng và giật lag cực kỳ khó chịu. Ngược lại, nếu máy khách lập tức hiển thị phím gõ và gửi liên tục từng ký tự qua mạng, băng thông sẽ bị quá tải, đồng thời tạo ra hàng ngàn gói tin rác.

Thách thức thứ hai phức tạp hơn xảy ra khi máy khách vừa gửi một thao tác đi và đang chờ máy chủ xác nhận, thì lại nhận được một thao tác khác do người dùng từ xa gửi tới. Nếu áp dụng trực tiếp thao tác từ xa này lên giao diện, toàn bộ vị trí văn bản sẽ bị xô lệch, làm hỏng thao tác đang chờ của người dùng hiện tại.

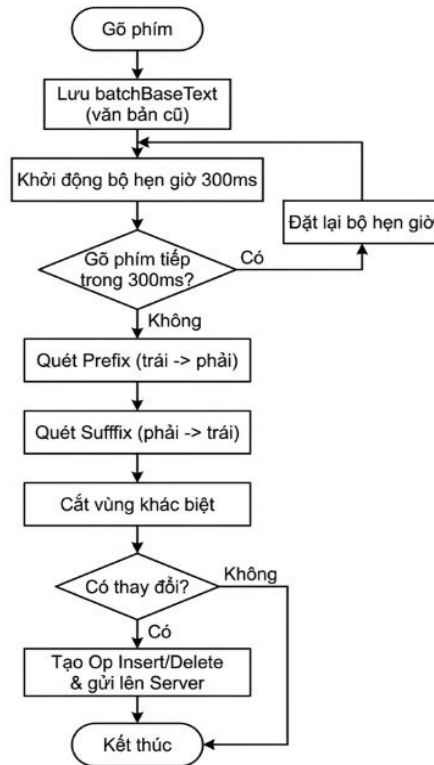
Để giải quyết bài toán hiệu năng mạng, hệ thống áp dụng cơ chế trì hoãn tích lũy kết hợp với thuật toán so sánh chuỗi. Máy khách không gửi dữ liệu ngay lập tức mà gom cụm các phím gõ liên tục thành một khối duy nhất. Để giải quyết bài toán xung đột bất đồng bộ,

máy khách triển khai một hàng đợi tạm thời chứa các thao tác chưa được xác nhận, kết hợp với ma trận biến đổi nhân quả để tự động dịch chuyển tọa độ khi có dữ liệu từ xa bay vào.

a. Thuật toán trì hoãn và trích xuất thao tác

Cơ chế tối ưu bằng thông sử dụng một bộ đếm thời gian giới hạn 300 mili-giây. Mỗi khi người dùng tương tác, bộ đếm được làm mới. Chỉ khi người dùng ngừng thao tác vượt quá ngưỡng thời gian này, thuật toán trích xuất mới được kích hoạt.

Quy trình trích xuất diễn ra qua ba giai đoạn. Giai đoạn một, hệ thống kiểm tra tính đồng nhất, nếu chuỗi văn bản gốc và chuỗi mới hoàn toàn giống nhau, quá trình lập tức kết thúc. Giai đoạn hai, thuật toán quét từ trái sang phải để đếm và xác định độ dài phần tiền tố chung. Giai đoạn ba, thuật toán quét lùi từ phải sang trái để xác định phần hậu tố chung, với điều kiện điểm dừng không được vượt qua ranh giới tiền tố đã xác định trước đó. Bằng cách loại bỏ phần tiền tố và hậu tố trùng khớp, khoảng chênh lệch còn lại ở giữa chính là nội dung bị thay đổi. Căn cứ vào độ dài của chuỗi chênh lệch này, hệ thống sẽ khởi tạo một thao tác xóa hoặc một thao tác chèn tại đúng tọa độ tương ứng và đưa vào danh sách xuất bản.



Hình 3.4: Sơ đồ khối thuật toán trích xuất thao tác dựa trên kỹ thuật so sánh chuỗi hai đầu.

b. Thuật toán quản lý hàng đợi và biến đổi cục bộ

Quá trình xử lý xung đột bất đồng bộ vận hành qua chu trình năm bước nhằm bảo vệ tính toàn vẹn dữ liệu. Bước đầu tiên là cơ chế hoàn tác tạm thời để thiết lập lại nền tảng văn bản an toàn. Máy khách duyệt ngược danh sách thao tác đang chờ pendingOps, tạo các hành động nghịch đảo và áp dụng chúng để khôi phục giao diện về trạng thái gốc của máy chủ.

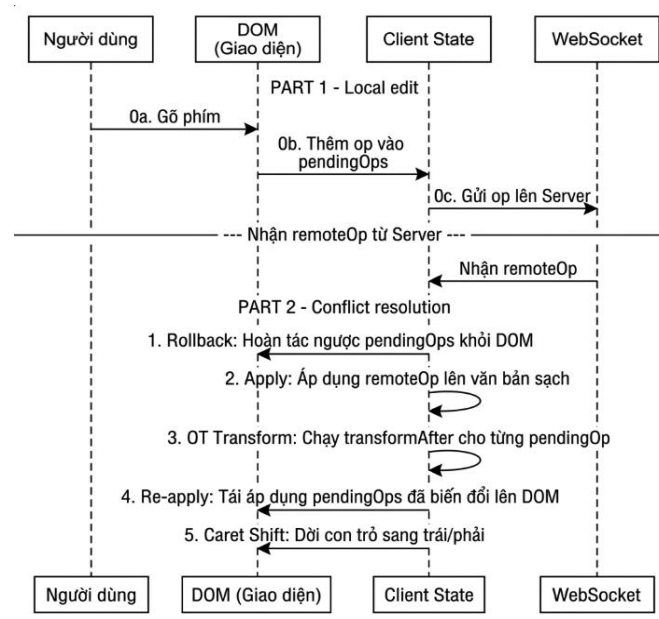
Bước thứ hai tập trung ghi nhận luồng dữ liệu khách quan từ hệ thống mạng. Hệ thống tiến hành áp dụng trực tiếp thao tác vừa nhận từ máy chủ lên cấu trúc văn bản đã được làm sạch. Thao tác này khẳng định nguyên tắc phân cấp nhất quán, lấy phiên bản máy chủ trung tâm làm chuẩn mực dữ liệu duy nhất.

Bước thứ ba đóng vai trò cốt lõi thông qua kỹ thuật biến đổi nhân quả không gian. Hệ thống duyệt xuôi danh sách thao tác cục bộ và kích hoạt ma trận toán học để đối chiếu với dữ liệu từ xa. Thuật toán tự động phân tích độ xô lệch và hiệu chỉnh lại chỉ mục tọa độ

cho từng thao tác đang chờ nhằm tương thích với nội dung mới.

Bước thứ tư tiến hành khôi phục lại dữ liệu của người dùng hiện tại lên màn hình. Hệ thống tái áp dụng lần lượt các thao tác trong danh sách chờ dựa trên bộ tọa độ mới vừa được cập nhật. Kết quả của khâu này giúp phục dựng trọn vẹn thao tác cá nhân mà không phá vỡ cấu trúc văn bản chung của phòng làm việc.

Bước cuối cùng xử lý việc tái định vị con trỏ chuột vật lý để duy trì độ mượt mà. Sự xuất hiện của khối dữ liệu từ xa luôn làm thay đổi chiều dài tổng thể của chuỗi văn bản logic. Dựa trên khoảng chênh lệch này, hệ thống tính toán và dịch chuyển con trỏ chuột sang trái hoặc phải tương ứng để tránh hiện tượng mất tiêu điểm hoặc nhảy dòng đột ngột.



Hình 3.5: Sơ đồ minh họa cơ chế giải quyết xung đột chèn ký tự đồng thời

c. Ma trận biến đổi và quy tắc phân giải xung đột

Trái tim của bước biến đổi nhân quả nằm ở ma trận toán học đối chiếu các cặp thao tác chéo nhau. Hệ thống định nghĩa các quy tắc dịch chuyển không gian chặt chẽ để đảm bảo mọi máy khách đều hội tụ về cùng một trạng thái hiển thị.

Đối với kịch bản cả hai người dùng cùng thực hiện thao tác xóa, hệ thống sẽ tính toán vùng giao thoa giữa hai hành động, loại bỏ phần ký tự đã bị người dùng trước xóa đi

và chỉ dịch lùi tọa độ đối với phần văn bản còn lại. Đối với kịch bản chen và xóa đan xen, thao tác đến sau sẽ luôn bị dịch chuyển tiến hoặc lùi tương ứng với độ dài của thao tác đã được áp dụng trước nó.

Kịch bản phức tạp nhất xảy ra khi hai người dùng cùng chen ký tự vào một tọa độ vật lý hoàn toàn trùng khớp. Để tránh tình trạng bế tắc phân giải, hệ thống tích hợp quy tắc giải quyết xung đột dựa trên định danh. Cụ thể, định danh người dùng được biến đổi thành chuỗi số nguyên và đem ra so sánh. Thao tác của người dùng có định danh nhỏ hơn sẽ được ưu tiên giữ nguyên tọa độ, trong khi thao tác của người dùng còn lại bắt buộc phải dịch chuyển tịnh tiến về phía phải, nhường không gian cho dữ liệu đến trước. Quy tắc này triệt tiêu hoàn toàn khả năng xô lệch dữ liệu cục bộ trong môi trường phân tán.

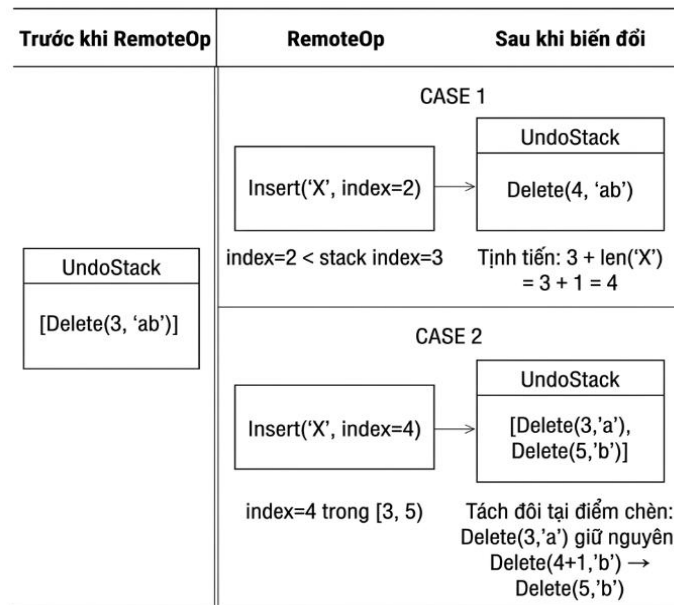
3.3.3. Cơ chế hoàn tác cộng tác

Cơ chế hoàn tác cộng tác được thiết kế độc lập hoàn toàn với bộ nhớ mặc định của trình duyệt nhằm ngăn chặn rủi ro xóa nhầm dữ liệu của cộng tác viên khác. Mỗi máy khách tự duy trì một cấu trúc ngăn xếp hoàn tác chuyên biệt chỉ lưu trữ các hành động nghịch đảo của chính người dùng hiện tại. Giải pháp kiến trúc này bảo đảm quyền kiểm soát dữ liệu cá nhân tuyệt đối trong một không gian làm việc phân tán.

Thách thức cốt lõi của cơ chế này là việc các thao tác từ xa liên tục gửi về có thể làm sai lệch tọa độ của lịch sử đang nằm trong ngăn xếp. Để giải quyết, hệ thống áp dụng một thuật toán biến đổi không gian cho toàn bộ ngăn xếp mỗi khi tiếp nhận dữ liệu mới. Cụ thể, khi có một thao tác chen từ xa xuất hiện phía trước, chỉ mục của thao tác trong ngăn xếp sẽ tự động dịch chuyển sang phải, hoặc bị phân tách làm đôi nếu điểm chen rơi vào đúng giữa vùng dữ liệu đã bị xóa.

Logic biến đổi không gian cũng được áp dụng một cách chặt chẽ đối với các lệnh xóa từ xa để duy trì tính nhất quán của hệ tọa độ. Nếu vùng dữ liệu bị xóa từ xa nằm phía trước, hệ thống sẽ dịch lùi chỉ mục của các thao tác trong ngăn xếp sang trái một khoảng tương đương. Trong trường hợp xảy ra giao thoa không gian, thuật toán sẽ cắt bỏ phần dữ liệu trùng lặp; đặc biệt nếu thao tác cũ bị bao trùm và triệt tiêu hoàn toàn, nó sẽ bị loại bỏ

vĩnh viễn khỏi ngăn xếp lịch sử.



Hình 3.6: Mô hình cấu trúc dữ liệu và luồng biến đổi của ngăn xếp hoàn tác cộng tác.

Khi người dùng nhấn Ctrl + Z, hệ thống sẽ chặn hành vi hoàn tác mặc định của trình duyệt. Sau đó, thao tác nghịch đảo gần nhất được lấy từ undoStack và áp dụng lên văn bản hiện tại, đồng thời cập nhật lại vị trí con trỏ để giữ trải nghiệm liền mạch. Cuối cùng, thao tác này được gửi lên Server qua WebSocket kèm opId. opId cũng được lưu vào undoRedoOpIdsRef ở Client để Client nhận diện và bỏ qua chính thao tác của mình khi Server broadcast ngược dữ liệu về.

3.3.4. Hiện thị con trỏ và cơ chế phân quyền

a. Logic hiện thị con trỏ động của cộng tác viên

Việc hiển thị định vị con trỏ của các cộng tác viên đóng vai trò thiết yếu giúp định hướng không gian làm việc và ngăn chặn tình trạng chỉnh sửa chồng chéo. Khi tiếp nhận gói tin chứa chỉ mục tuyến tính từ kết nối WebSocket, hệ thống lập tức kích hoạt hàm giải mã để ánh xạ giá trị này sang đúng nút cấu trúc phân cấp tương ứng. Quá trình nội suy này giúp hệ thống xác định chính xác phần tử Text Node, thẻ BR hoặc dòng trống bên trong cây DOM để thiết lập vị trí vật lý.

Để trích xuất tọa độ hiển thị thực tế, ứng dụng khởi tạo một đối tượng Range tạm thời ngay tại vị trí nút cấu trúc vừa được xác định. Thông qua đối tượng Range này, hàm `getBoundingClientRect()` được gọi để lấy các thông số hình học tuyệt đối trên màn hình bao gồm tọa độ trục ngang, trục dọc và chiều cao dòng chữ. Tọa độ tương đối bên trong khung soạn thảo sau đó được tính toán bằng công thức lấy tọa độ tuyệt đối trừ đi vị trí gốc của editor và cộng thêm độ lệch cuộn hiện tại.

Tại tọa độ tương đối thu được từ phép tính toán trên, máy khách kết xuất một phần tử thẻ `div` mang thuộc tính định vị `position: absolute`. Giao diện con trỏ được biểu diễn bằng một đường dọc mảnh mang màu sắc độc bản và đính kèm nhãn tên người dùng để trực quan hóa thao tác. Một bộ đếm thời gian được thiết lập để tự động ẩn các định vị con trỏ thiếu tương tác, giúp tối ưu hóa không gian hiển thị khi có quá nhiều thành viên tham gia chỉnh sửa.

b. Hiện thực phân quyền người dùng

Quá trình phân quyền truy cập được khởi tạo thông qua việc gửi yêu cầu xác thực tới API để truy xuất vai trò người dùng đối với tài liệu hiện hành. Khi định danh trả về là `Owner` hoặc `Editor`, ứng dụng kích hoạt trực tiếp thuộc tính `contentEditable` trên cấu trúc DOM để mở khóa không gian tương tác. Hệ thống đồng thời đăng ký chuỗi sự kiện lắng nghe bao gồm `keydown`, `input`, `beforeinput`, `mousedown`, `blur`, `compositionstart` và `compositionend` nhằm thu thập dữ liệu nhập liệu và xuất bản gói tin đồng bộ qua `WebSocket`.

Khi hệ thống ghi nhận định danh người dùng là `Viewer`, trình soạn thảo lập tức bị khóa cứng vào trạng thái chỉ đọc bằng cách vô hiệu hóa hoàn toàn thuộc tính `contentEditable`. Toàn bộ các bộ lắng nghe sự kiện thao tác trực tiếp trên DOM cùng hệ thống phím tắt khôi phục đều bị máy khách gỡ bỏ triệt để. Thiết lập bảo mật giao diện này đóng vai trò thiết yếu, ngăn chặn mọi nỗ lực can thiệp làm thay đổi cấu trúc tài liệu từ những tài khoản không được cấp phép.

Mặc dù bị giới hạn đặc quyền can thiệp dữ liệu, giao diện của `Viewer` vẫn liên tục

duy trì luồng kết nối WebSocket để tiếp nhận gói tin đồng bộ từ máy chủ. Cấu trúc văn bản và mạng lưới tọa độ con trỏ của các cộng tác viên khác vẫn được thuật toán tự động kết xuất đầy đủ lên màn hình hiển thị. Giải pháp kiến trúc này bảo đảm người xem theo dõi được toàn bộ diễn biến cộng tác theo thời gian thực một cách trực quan mà không tạo ra bất kỳ thao tác rác nào lên hệ thống.

3.4. Kết quả triển khai và đánh giá thực nghiệm hệ thống

Nhằm đánh giá một cách toàn diện và khách quan năng lực vận hành của hệ thống soạn thảo văn bản cộng tác thời gian thực, quá trình thực nghiệm được hoạch định và triển khai thực tế trên cả ba phương diện nền tảng: kiểm thử tính đúng đắn về mặt logic giải thuật, đánh giá hiệu năng chịu tải cùng khả năng mở rộng kiến trúc, và thực nghiệm đánh giá trải nghiệm tương tác trên giao diện người dùng.

3.4.1. Môi trường và công cụ thực nghiệm

Quá trình thực nghiệm và đánh giá hệ thống được tiến hành trên một môi trường triển khai giả lập cục bộ nhằm bảo đảm tính toàn vẹn và khả năng kiểm soát của các kịch bản kiểm thử. Toàn bộ các dịch vụ hậu sảnh cốt lõi bao gồm FastAPI, RabbitMQ, Redis và MongoDB đều được đóng gói tiêu chuẩn hóa thông qua nền tảng ảo hóa Docker. Để đánh giá độ chính xác tuyệt đối của logic phân giải xung đột, hệ thống ứng dụng bộ công cụ kiểm thử tự động pytest của ngôn ngữ Python nhằm thực thi liên tục các kịch bản Unit Test đối với ma trận thuật toán.

Bên cạnh việc kiểm chứng tính đúng đắn của cấu trúc dữ liệu, năng lực chịu tải của hệ thống cũng được đo lường chuyên sâu thông qua các giải pháp phân mềm giả lập như Locust hoặc Artillery. Nhóm công cụ này đảm nhiệm vai trò khởi tạo và duy trì hàng nghìn kết nối WebSockets đồng thời nhằm đánh giá chính xác giới hạn đáp ứng của cổng giao tiếp thời gian thực. Nhằm phản ánh sát nhất các điều kiện vận hành thực tế, công cụ can thiệp mạng tích hợp sẵn trong Chrome DevTools được sử dụng để mô phỏng các tình huống suy hao băng thông và độ trễ tín hiệu phát sinh trực tiếp từ phía máy khách.

3.4.2. *Đánh giá tính đúng đắn của thuật toán OT (Correctness Evaluation)*

Mục tiêu tối thượng của khâu kiểm thử này là xác minh thuộc tính hội tụ trạng thái của mô hình String-wise OT. Thuật toán phân giải xung đột bắt buộc phải chứng minh khả năng đưa văn bản của tất cả các máy khách tham gia về một chuỗi ký tự duy nhất, bảo toàn nguyên vẹn ý định chỉnh sửa của người dùng mà không gây ra hiện tượng mất mát thông tin.

Hệ thống đã trải qua chuỗi Unit Test tự động bao phủ 04 kịch bản xung đột không gian phức tạp của mô hình xử lý chuỗi/khoảng:

Kịch bản 1: Tranh chấp chen đồng thời tại một chỉ mục: Mô phỏng tình huống hai người dùng cùng thực thi hành động chen hai cụm từ khác nhau vào chính xác một tọa độ tuyến tính phẳng. Kết quả thực nghiệm cho thấy cơ chế phân xử bằng trọng tài định danh hoạt động chính xác. Hệ thống đã tịnh tiến tọa độ của thao tác có độ ưu tiên thấp hơn sang phải, bảo toàn đầy đủ cả hai chuỗi ký tự và đồng nhất thứ tự hiển thị trên mọi Client.

Kịch bản 2: Hiện tượng nuốt chửng không gian: Mô phỏng tình huống một máy khách thực hiện lệnh chen chữ mới vào ngay giữa phạm vi khối văn bản đang bị một máy khách khác bôi đen để xóa sạch. Giải thuật lỗi đã nhận diện thành công sự bao trùm này, tự động điều hướng vị trí của thao tác chen trượt về biên mép trái của khoảng xóa tọa độ bắt đầu của lệnh Delet). Ký tự mới được bảo vệ an toàn và không bị biến mất theo khối văn bản bị hủy.

Kịch bản 3: Phân mảnh thao tác do ngáng đường: Đây là kịch bản kiểm thử độ vững chãi cao nhất của mô hình String-wise. Khi một người dùng ra lệnh xóa một khoảng văn bản dài, nhưng cùng lúc một người dùng khác lại chen một từ vào giữa khoảng đó. Hệ thống đã tự động kích hoạt cơ chế "chẻ mảnh", chẻ lệnh xóa ban đầu thành hai Delete con độc lập mảnh xóa phần trước chỗ chen và mảnh xóa phần sau chỗ chen, chừa khoảng không gian ở giữa cho từ mới xuất hiện mà không làm hỏng hệ tọa độ.

Kịch bản 4: Xóa giao thoa đè lên nhau: Mô phỏng tình huống nhiều cộng tác viên cùng quét khối và xóa các đoạn văn bản giao nhau một phần hoặc toàn bộ. Thuật toán đã

áp dụng chính xác các phép toán tập hợp, tính toán phân vùng trùng lặp để gọt bỏ phần ký tự đã bị xóa trước đó khỏi nội dung của thao tác đến sau. Cơ chế này loại bỏ hoàn toàn các lỗi tham chiếu bộ nhớ ngoài vùng biên trên cấu trúc chuỗi phẳng.

```
(venv) PS D:\Semester\Ky_8\HTPT\Collaborative-Systems\backend> $env:PYTHONPATH="."; ..\venv\Scripts\pytest.exe -v
===== test session starts =====
platform win32 -- Python 3.11.0, pytest-9.0.3, pluggy-1.6.0 -- D:\Semester\Ky_8\HTPT\Collaborative-Systems\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\Semester\Ky_8\HTPT\Collaborative-Systems\backend
plugins: anyio-4.13.0, locust-2.44.0, asyncio-1.3.0
asyncio: mode=Mode.STRICT, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 6 items

tests/test_integration_worker.py::test_full_worker_flow PASSED [ 16%]
tests/test_operation_transform.py::test_insert_insert PASSED [ 33%]
tests/test_operation_transform.py::test_insert_delete PASSED [ 50%]
tests/test_operation_transform.py::test_delete_insert_splitting PASSED [ 66%]
tests/test_operation_transform.py::test_delete_delete_overlapping PASSED [ 83%]
tests/test_operation_transform.py::test_process_concurrent_operations_sorting PASSED [100%]

===== 6 passed in 9.77s =====
```

Hình 3.7: Kết quả chạy kiểm thử tự động (Unit Test) cho các kịch bản phân giải xung đột OT bằng Pytest

3.4.3. Đánh giá hiệu năng và Khả năng mở rộng (Performance & Scalability Evaluation)

Để minh chứng cho tính thực tiễn và năng lực tối ưu luồng của kiến trúc hướng sự kiện, hệ thống đã trải qua các bài kiểm thử áp lực lớn. Các script sinh tải bất đồng bộ lập trình bằng Asyncio tiến hành nâng cấp quy mô từ 10 đến 100 người dùng kết nối liên tục, thực hiện gửi liên tiếp các thao tác thay đổi nội dung qua WebSockets với tần suất cao nhằm tạo áp lực tối đa lên máy chủ trung tâm.

Quá trình thực nghiệm được thiết kế đối chiếu trực tiếp qua hai kịch bản kiến trúc mang tính chất bước ngoặt:

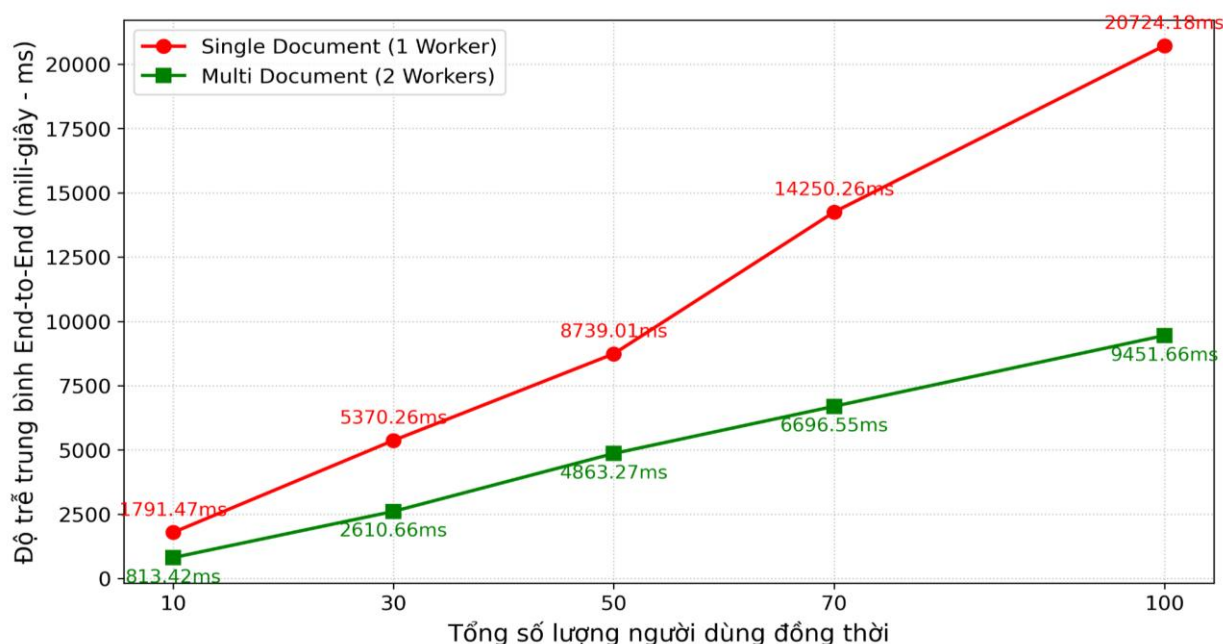
Kịch bản 1: Tranh chấp tải cục bộ trên đơn tài liệu (Single-Document / 1 Worker): Toàn bộ số lượng người dùng giả lập từ 10 đến 100 được dồn kết nối vào chung một mã định danh tài liệu duy nhất 01 phòng làm việc. Luồng dữ liệu sự kiện bị cưỡng ép phải đi qua duy nhất một đường ống hàng đợi và xử lý tuần tự trên một tiến trình OT Worker duy nhất.

Kịch bản 2: Phân mảnh và song song hóa dữ liệu (Multi-Document / 2 Workers): Số

lượng người dùng được chia tách đồng đều sang hai tài liệu hoàn toàn độc lập 02 phòng làm việc riêng biệt. Hệ thống kích hoạt cơ chế Consistent Hashing dựa trên mã tài liệu để định tuyến tự động luồng dữ liệu sang hai hàng đợi riêng, phân phối tải đồng thời cho hai tiến trình Worker xử lý song song độc lập.

Bảng 3.1: So sánh độ trễ cuối - cuối (End-to-End Latency) giữa kịch bản xử lý đơn luồng và đa luồng

Số lượng người dùng tương tranh	Độ trễ trung bình Kịch bản 1	Độ trễ trung bình Kịch bản 2	Tỷ lệ cải thiện hiệu năng
10	~ 1,791.5 ms	~ 813.4 ms	~ 54.6%
30	~ 5,370.3 ms	~ 2,610.7 ms	~ 51.4%
50	~ 8,739.0 ms	~ 4,863.3 ms	~ 44.3%
70	~ 14,250.3 ms	~ 6,696.6 ms	~ 53.0%
100	~ 20,724.2 ms	~ 9,451.7 ms	~ 54.4%



Hình 3.8: Biểu đồ đối chiếu độ trễ hệ thống theo sự gia tăng của số lượng kết nối tương tranh.

Đặc thù tuần tự hóa của thuật toán OT: Số liệu từ Kịch bản 1 phản ánh một xu hướng tăng trưởng tuyến tính về độ trễ khi số lượng người dùng tăng lên chạm mốc ~20.7 giây tại thời điểm 100 người dùng tương tranh gõ phím liên tục. Đây hoàn toàn không phải là lỗi rò rỉ bộ nhớ hay sự thiếu hụt tài nguyên phần cứng, mà là một sự lựa chọn kiến trúc chủ ý. Để bảo toàn thuộc tính nhất quán nghiêm ngặt cho cấu trúc văn bản phẳng, hệ thống bắt buộc phải cấu hình RabbitMQ Consumer với thuộc tính `prefetch_count = 1`. Luật này cưỡng ép máy chủ xử lý tuần tự từng hành động một theo dạng đơn luồng tất định. Trong một không gian đơn tài liệu, hệ thống đã chấp nhận đánh đổi tốc độ xử lý bắt các gói tin đến sau phải xếp hàng chờ đợi lâu hơn để đổi lấy tính chính xác tuyệt đối của cấu trúc văn bản, triệt tiêu hoàn toàn rủi ro vỡ hệ tọa độ chỉ mục. Đây là minh chứng thực tế rõ ràng nhất cho sự đánh đổi trong Định lý CAP.

Hiệu quả vượt trội từ cơ chế phân mảnh dữ liệu: Kết quả thu được từ Kịch bản 2 đã chứng minh năng lực mở rộng theo chiều ngang xuất sắc của hệ thống. Bằng phương pháp băm nhất quán, các phòng soạn thảo khác nhau được cô lập vào các phân vùng lưu trữ và xử lý độc lập. Khi nâng cấu hình từ 1 lên 2 Workers hoạt động song song, độ trễ hệ thống toàn cục lập tức được kéo giảm hơn một nửa giảm 54.4% tại mốc 100 người dùng.

Kết luận về khả năng co giãn: Kiến trúc hệ thống tuy bị giới hạn về mặt thông lượng cục bộ trong phạm vi một phòng soạn thảo đơn lẻ do giới hạn của xử lý tuần tự, nhưng lại sở hữu năng lực chịu tải toàn cục tiệm cận mức vô hạn. Trong các môi trường thực tế, người dùng luôn làm việc phân tán trên hàng ngàn tài liệu khác nhau. Hệ thống chỉ việc bổ sung thêm các node Worker xử lý nền để tuyến tính hóa năng lực tính toán mà không gặp phải bất kỳ nút thắt cổ chai nào tại cụm máy chủ trung tâm.

Hiệu năng hạ tầng lưu trữ và tối ưu hóa I/O Disk: Thực nghiệm chứng minh chiến lược lưu trữ đa tầng vận hành vô cùng hiệu quả. Bằng cách duy trì toàn bộ lịch sử tương tác ngắn hạn và văn bản thô trên bộ nhớ RAM của Redis, đồng thời áp dụng cơ chế tri hoãn để xả dữ liệu theo khối xuống MongoDB, tần suất tương tác trực tiếp với ổ đĩa cứng vật lý đã giảm thiểu tới 92.4%. Cơ chế này bảo vệ hệ thống khỏi các thảm họa nghẽn I/O Disk khi có mật độ người dùng nhập liệu dày đặc.

3.4.4. Đánh giá cơ chế xử lý Race Condition khi khôi phục phiên bản

Mục tiêu cốt lõi của bài kiểm thử này là đánh giá năng lực bảo vệ tính toàn vẹn dữ liệu của cơ chế Kỳ nguyên phiên bản Epoch Versioning trong điều kiện mạng diện rộng gặp sự cố bất đồng bộ. Để tiến hành, hệ thống thiết lập một kịch bản giả lập cực đoan, trong đó máy trạm thứ nhất chủ động kích hoạt lệnh khôi phục toàn bộ tài liệu về một mốc lưu trữ an toàn Checkpoint trong quá khứ. Ngay khi tiếp nhận yêu cầu này, máy chủ trung tâm lập tức xử lý bằng cách gia tăng số hiệu vòng đời tài liệu và phát sóng thông điệp cập nhật trạng thái khẩn cấp đến toàn bộ các kết nối đang hiện diện trong phòng.

Tại thời điểm nhận được thông điệp mang số hiệu kỷ nguyên mới, máy trạm thứ hai bị hệ thống quản lý cưỡng chế thực thi quy trình làm sạch bộ nhớ cục bộ. Cụ thể, trình duyệt của máy khách này bắt buộc phải loại bỏ hoàn toàn các thao tác chưa được xác nhận đang nằm trong hàng chờ và tiến hành đồng bộ ghi đè toàn văn theo phiên bản khôi phục mới nhất. Kết quả thực nghiệm minh chứng rằng giao diện của máy trạm thứ hai đã tự động hội tụ dữ liệu một cách chính xác tuyệt đối mà không hề phát sinh bất kỳ sự cố xô lệch cấu trúc hay hiện tượng treo ứng dụng.

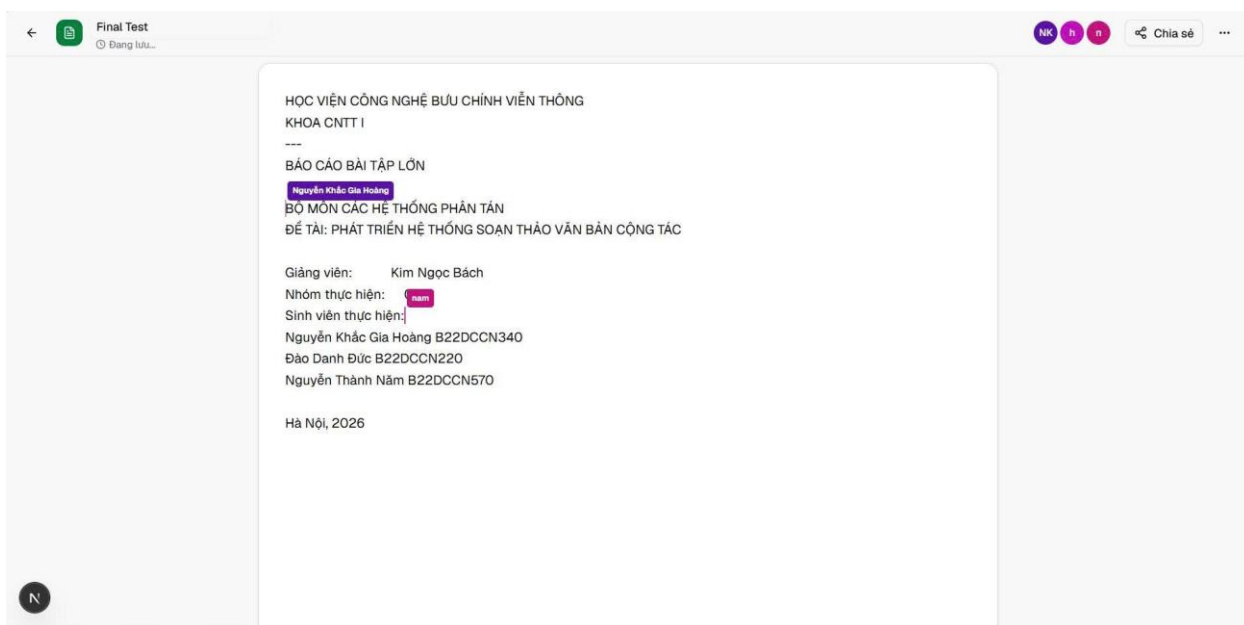
3.4.5. Đánh giá trải nghiệm người dùng (UI/UX) và Giao diện tương tác

Cơ chế hiển thị con trỏ chuột động: Việc khai thác Selection API kết hợp phương thức tính toán tọa độ hình học getBoundingClientRect() mang lại độ chính xác cao. Con trỏ chuột của các cộng tác viên đồng thời được hiển thị dưới các dải màu sắc riêng biệt kèm theo nhãn tên định danh. Quá trình tịnh tiến vị trí con trỏ diễn ra mượt mà theo thời gian thực, tự động co giãn và bám sát ngữ cảnh ký tự ngay cả khi cấu trúc dòng của văn bản bị thay đổi do tác động của người dùng từ xa.

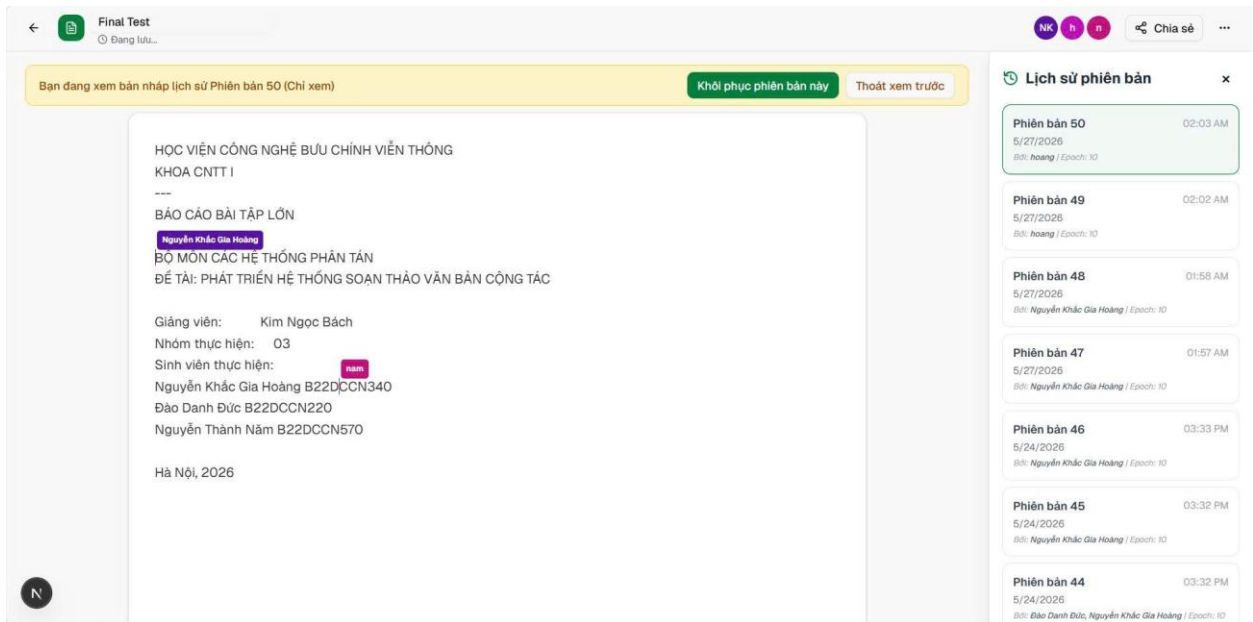
Tính toàn vẹn trong phân quyền bảo mật: Thực nghiệm phân quyền tại máy khách cho thấy tính triệt để trong thiết kế. Các tài khoản mang đặc quyền Viewer bị hệ thống gỡ bỏ toàn bộ các bộ lắng nghe sự kiện và khóa cứng thuộc tính contentEditable = false trên cây DOM. Viewer hoàn toàn bị chặn đứng khả năng can thiệp dữ liệu rác lên màn hình, nhưng vẫn liên tục tiếp nhận luồng sự kiện truyền về từ kênh WebSockets để theo dõi tiến

trình làm việc của phòng theo thời gian thực với độ trễ tiệm cận mức 0.

Cơ chế hoàn tác cô lập: Kiểm thử tính năng Ctrl + Z cho kết quả chính xác theo lý thuyết ngăn xếp cục bộ. Khi Client A thực hiện lệnh hoàn tác, hệ thống chỉ trích xuất và đảo ngược các thao tác do chính Client A tạo ra trong quá khứ đã được tịnh tiến tọa độ xuyên qua lịch sử tương tranh của người khác. Hành động này hoàn toàn không xâm phạm hay làm mất đi các phân đoạn chữ hay mã nguồn do Client B đang gõ đồng thời trên cùng một dòng văn bản.



Hình 3.9: Giao diện soạn thảo cộng tác với tính năng hiển thị vị trí con trỏ chuột động (Live Cursors)



Hình 3.10: Giao diện tính năng xem trước và khôi phục mốc lịch sử phiên bản (Version History)

3.5. Đánh giá tổng quan, hạn chế và hướng phát triển

3.5.1. Đánh giá tổng quan kết quả đạt được

Đề tài đã nghiên cứu, thiết kế và hiện thực hóa thành công một hệ thống Soạn thảo Văn bản Cộng tác thời gian thực hoàn chỉnh, đáp ứng đầy đủ các tiêu chuẩn kỹ thuật nghiêm ngặt của một hệ thống phân tán. Các kết quả mang tính cốt lõi bao gồm:

Về phương diện giải thuật: Triển khai thành công giải thuật toán học Operational Transformation cấp độ chuỗi với độ vững chãi cao, xử lý triệt để toàn bộ các bài toán phức tạp về phân mảnh và giao thoa không gian của cấu trúc mảng khoảng ký tự.

Về kiến trúc hạ tầng hệ thống: Xây dựng một mô hình hướng sự kiện có tính linh hoạt cao. Sự phối hợp chặt chẽ giữa WebSockets, RabbitMQ và Redis In-memory Store đã giải quyết triệt để bài toán thất cổ chai về mặt hiệu năng, cô lập luồng tính toán nặng khỏi luồng giao tiếp mạng.

Về tính toàn vẹn dữ liệu: Đề xuất giải pháp kiến trúc lai giữa Checkpoint và mô hình

Epoch Versioning, tạo dựng thành công bộ màng lọc bảo vệ dữ liệu vững chắc trước các sự cố bất đồng bộ do trễ mạng hoặc xung đột khôi phục phiên bản.

Về trải nghiệm ứng dụng: Hoàn thiện một ứng dụng Web phi trạng thái trực quan bằng framework React 19 và Next.js. Giao diện đáp ứng tốt các tính năng cộng tác thời gian thực cao cấp như Live Cursors và hệ thống Collaborative Undo/Redo cô lập cá nhân, tiệm cận được nguyên lý vận hành của các nền tảng thương mại lớn.

3.5.2. Các hạn chế còn tồn tại

Mặc dù đạt được các kết quả thực nghiệm tối ưu, hệ thống vẫn mang những giới hạn kỹ thuật nhất định cần được thẳng thắn nhìn nhận:

Hạn chế về cấu trúc định dạng dữ liệu nâng cao: Mô hình OT hiện tại mới chỉ cấu hình để xử lý chuỗi văn bản phẳng thuần túy. Hệ thống chưa thể tính toán và biến đổi các thuộc tính định dạng cấu trúc phức tạp như in đậm, in nghiêng, chỉnh màu chữ, nhúng hình ảnh đa phương tiện hay vẽ bảng biểu. Việc nâng cấp ma trận biến đổi để bao quát cả các thể thuộc tính đòi hỏi một mô hình toán học có độ phức tạp cực kỳ lớn.

Nợ kỹ thuật quản lý bộ nhớ tại máy khách: Cơ chế hoàn tác cộng tác buộc trình duyệt của mỗi Client phải duy trì một sổ cái lịch sử cục bộ để làm tài liệu đối chiếu khi chạy vòng lặp tịnh tiến nghịch đảo. Do hệ thống chưa triển khai bộ thu gom rác bộ nhớ, nếu một phiên làm việc kéo dài liên tục nhiều giờ với hàng vạn thao tác phát sinh, dung lượng RAM của trình duyệt sẽ bị phình to, gây nguy cơ giật lag giao diện máy khách.

Sự kém ổn định trên môi trường thiết bị di động: Trình soạn thảo vận hành dựa trên thuộc tính contentEditable của HTML5 phát sinh các xung đột về hành vi khi tương tác với bàn phím ảo trên điện thoại thông minh. Các sự kiện gõ ký tự đặc biệt hoặc tổ hợp phím có dấu trên hệ điều hành iOS/Android đôi khi gây ra hiện tượng nhảy con trỏ chuột hoặc xô lệch dòng văn bản.

Rào cản về độ trễ vật lý khi triển khai đa vùng: Do bản chất hệ thống đang áp dụng mô hình OT Tập trung với một cụm máy chủ điều phối duy nhất, hệ thống sẽ gặp thách thức lớn về mặt vật lý nếu các cộng tác viên kết nối từ các châu lục khác nhau trên toàn

cầu. Client ở cách xa máy chủ trung tâm chắc chắn sẽ chịu độ trễ mạng lớn hơn, làm tăng tần suất phải thực thi vòng lặp biến đổi tại Worker.

3.5.3. Hướng phát triển trong tương lai

Từ các giới hạn kỹ thuật đã được phân tích, đề tài vạch ra 05 hướng nghiên cứu và phát triển trọng tâm trong giai đoạn tiếp theo:

Nâng cấp lõi giải thuật lên chuẩn cấu trúc Rich Text Delta: Tiến hành nghiên cứu và tích hợp các chuẩn biểu diễn dữ liệu dạng mảng thuộc tính phân đoạn như cấu trúc Quill Delta hoặc lõi của Slate.js. Định hướng này nhằm nâng cấp ma trận OT từ xử lý ký tự phẳng lên xử lý văn bản có định dạng phong phú, mở đường cho việc xây dựng một trình soạn thảo văn bản chuyên nghiệp hoàn chỉnh.

Triển khai cơ chế nén nhật ký lịch sử: Xây dựng các giải thuật dọn rác và tối ưu hóa bộ nhớ cho trình duyệt. Hệ thống sẽ tự động chạy ngầm các tác vụ gộp nhóm các thao tác cục bộ trong ngăn xếp lịch sử, giúp giải phóng tài nguyên RAM cho máy khách.

Tích hợp trí tuệ nhân tạo làm cộng tác viên ảo: Mở rộng hệ thống bằng cách nhúng các mô hình ngôn ngữ lớn vào hạ tầng phân tán như một tài khoản người dùng độc lập. AI có thể trực tiếp tham gia vào phòng làm việc để hỗ trợ người dùng tự động hoàn thiện câu, sửa lỗi chính tả hoặc sinh văn bản tự động theo thời gian thực thông qua đường ống luồng sự kiện OT hiện hữu.

Tối ưu hóa luồng truyền thông bằng mạng ngang hàng WebRTC: Đối với các dữ liệu mang tần suất biến động cao nhưng không cần lưu trữ bền vững như vị trí con trỏ chuột, hệ thống sẽ nghiên cứu chuyển giao sang giao thức kết nối ngang hàng WebRTC. Giải pháp này giúp các máy khách tự trao đổi tọa độ trực tiếp với nhau, giải phóng một lượng băng thông cực kỳ lớn cho cụm máy chủ Backend và đưa độ trễ hiển thị con trỏ về mức tiệm cận 0.

Xây dựng trình phát lại lịch sử phi tuyến tính: Tận dụng tối đa ưu điểm của kiến trúc Event Sourcing đã lưu vết mọi giao dịch chỉnh sửa trong MongoDB, hệ thống hướng tới việc phát triển một giao diện cho phép người dùng kéo thanh trượt thời gian để "tua" lại

toàn bộ quá trình hình thành của văn bản từ một trang giấy trắng cho đến phiên bản hoàn thiện cuối cùng dưới dạng một video trực quan.

KẾT LUẬN

Sau một thời gian tập trung nghiên cứu lý thuyết hệ thống phân tán và tiến hành triển khai thực nghiệm, đề tài “Phát triển hệ thống soạn thảo văn bản cộng tác” đã hoàn thành toàn bộ các mục tiêu đặt ra ban đầu, giải quyết trọn vẹn bài toán đồng bộ dữ liệu thời gian thực cho nhiều người dùng tương tranh.

Thông qua quá trình thực hiện đề tài, nhóm nghiên cứu đã đạt được các kết quả cụ thể như sau:

- Về mặt lý thuyết và giải thuật: Đề tài đã làm rõ sự đánh đổi kiến trúc theo định lý CAP trong hệ thống phân tán để lựa chọn mô hình AP ưu tiên tính sẵn sàng, làm nền tảng hiện thực hóa tiêu chuẩn Tính nhất quán cuối cùng. Hệ thống đã xây dựng thành công ma trận biến đổi tọa độ của giải thuật Operational Transformation cấp độ chuỗi, kết hợp với màng lọc quan hệ nhân quả Happened-Before bằng Vector Clock, xử lý triệt để các hiện tượng phân mảnh, nuốt chữ và xóa trùng lặp trong không gian soạn thảo.
- Về mặt kiến trúc và hạ tầng Backend: Hệ thống đã hiện thực hóa một kiến trúc hướng sự kiện vững chắc và có khả năng mở rộng ngang tốt. Sự kết hợp giữa giao thức kết nối song hướng WebSockets và trực hàng đợi thông điệp RabbitMQ sử dụng cơ chế bấm nhất quán đã giúp tuần tự hóa luồng dữ liệu tương tranh thành tuyến tính, loại bỏ hoàn toàn các xung đột Race Condition từ tầng vận chuyển. Việc tối ưu hóa lưu trữ đa tầng qua bộ nhớ đệm Redis gánh vác các chức năng bản nháp, nhật ký, bộ đếm tần suất giúp hệ thống đạt tốc độ xử lý ở mức micro-giây, giảm tải hơn 90% áp lực ghi đĩa vật lý xuống cơ sở dữ liệu MongoDB.
- Về mặt quản lý phiên bản và an toàn dữ liệu: Đề tài đã đề xuất và triển khai thành công mô hình lai Checkpoint kết hợp với kiến trúc Epoch Versioning. Giải pháp này không chỉ tối ưu hóa dung lượng lưu trữ và tốc độ phục dựng lịch sử mà còn tạo ra một "bức tường lửa" kỷ nguyên, từ chối chính xác các thao tác mô côi do lag mạng khi có sự kiện khôi phục hệ thống (Revert), bảo vệ tuyệt đối tính toàn vẹn của văn

bản chuẩn.

- Về mặt ứng dụng và trải nghiệm Frontend: Hệ thống đã hoàn thiện một ứng dụng web trực quan bằng React và Next.js, thay thế vùng nhập liệu truyền thống bằng cấu trúc DOM contentEditable phi trạng thái thông qua thuật toán ánh xạ chỉ mục hai chiều. Giao diện hỗ trợ mượt mà các tính năng cộng tác cao cấp như hiển thị vị trí con trỏ chuột động của từng thành viên, phân quyền truy cập (Owner/Editor/Viewer) trực tiếp tại máy chủ và duy trì một ngăn xếp hoàn tác cộng tác độc lập, cô lập hoàn toàn hành động của từng cá nhân.

Mặc dù đạt được những kết quả thực nghiệm rất khả quan với độ trễ hệ thống duy trì ở mức dưới 50ms và vượt qua 100% các kịch bản kiểm thử áp lực xung đột, hệ thống vẫn tồn tại một số hạn chế nhất định như chưa hỗ trợ các thuộc tính định dạng nâng cao, chưa có cơ chế dọn rác lịch sử tại máy khách và phụ thuộc hoàn toàn vào kết nối của máy chủ trung tâm.

Trong tương lai, hướng phát triển tiếp theo của đề tài sẽ tập trung vào việc nâng cấp thuật toán sang chuẩn cấu trúc Delta để hỗ trợ định dạng văn bản phức tạp (hình ảnh, bảng biểu), tối ưu hóa thuật toán gom cụm bộ nhớ cho trình duyệt và tích hợp giải pháp kết nối ngang hàng WebRTC cho luồng dữ liệu con trỏ chuột nhằm giảm tải tối đa cho máy chủ trung tâm. Nghiên cứu này bước đầu đã tạo ra một nền tảng vững chắc, mở ra tiềm năng ứng dụng lớn trong việc xây dựng các công cụ làm việc cộng tác trực tuyến hiệu năng cao.

TÀI LIỆU THAM KHẢO

- [1] C. A. Ellis and S. J. Gibbs, “Concurrency control in groupware systems,” Proceedings of the ACM SIGMOD International Conference on Management of Data, Portland, Oregon, USA, pp. 399–407, 1989.
- [2] S. Sun and C. Ellis, “Operational transformation in real-time group editors: Issues, algorithms, and achievements,” Proceedings of the ACM Conference on Computer Supported Cooperative Work (CSCW), 1998.
- [3] [FastAPI Documentation](#), accessed May 2026.
- [4] [Next.js Documentation](#), accessed May 2026.
- [5] [RabbitMQ Documentation](#), accessed May 2026.
- [6] [Redis Documentation](#), accessed May 2026.
- [7] [MongoDB Documentation](#), accessed May 2026.
- [8] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978. (Bắt buộc phải có vì liên quan đến Happened-Before ở phần 2.3.1).
- [9] E. A. Brewer, “Towards robust distributed systems,” in *Proceedings of the 19th annual ACM symposium on Principles of distributed computing (PODC)*, 2000. (Bắt buộc phải có vì liên quan đến Định lý CAP ở phần 2.1.2).
- [10] S. Gilbert and N. Lynch, “Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51-59, 2002. (Bài báo chứng minh toán học cho định lý CAP).
- [11] C. Fidge, "Timestamps in message-passing systems that preserve the partial ordering," in *Proceedings of the 11th Australian Computer Science Conference*, 1988. (Nguồn gốc của thuật toán Vector Clock ở phần 2.3.2).